Minimizing weighted makespan in a two-machine flow shop

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Motivated by the scheduling of perishable lots in process industries, this paper studies the two-machine flow shop problem with the maximum weighted completion time objective, also known as the weighted makespan. We establish binary NP-hardness even with two distinct weights and strong NP-hardness when the number of distinct weights is arbitrary. We further prove that an optimal schedule has a block structure, with one block per distinct weight, blocks arranged in non-increasing order of weight, and jobs within each block sequenced according to Johnson's rule. Exploiting this structure, we develop an exact dynamic programming algorithm that is pseudo-polynomial for a fixed number of distinct weights, a fully polynomial-time approximation scheme for a fixed number of distinct weights, and a polynomial-time approximation scheme when the ratio of the largest to the smallest weight is bounded. We also propose a near-linear-time two-approximation algorithm and adapt the NEH heuristic and a weight-block genetic algorithm (WBGA) for larger instances. Computational experiments on 440 small-scale and 640 large-scale instances are conducted using a mixed-integer programming model and a lower bound as benchmarks. The results show that the two-approximation algorithm is the fastest method and remains closest to the lower bound on large instances. NEH offers the best quality-cost trade-off on small instances, whereas WBGA achieves the best solution quality on large instances at a substantially higher computational cost.

Keywords: Scheduling; Two-machine flow shop; Weighted makespan; Dynamic programming; Heuristic algorithm

*Corresponding author. E-mail address: author@email.com

1 Introduction

Scheduling decisions affect delivery performance, resource utilization, and a firm's capacity to meet heterogeneous customer requirements. The two-machine flow shop is a standard multi-stage scheduling model. Each job is processed on both machines in the same order. Examples include cells in which a bottleneck operation feeds finishing, assembly lines followed by testing, and process-industry settings with tightly coupled consecutive operations. Johnson's rule (Johnson, 1954) established that the makespan , i.e., the completion time of the last job, can be minimized in polynomial time for this problem. It remains a benchmark for flow-shop scheduling, and the structural insights behind it still guide algorithm design (Agnetis et al., 2025).

Makespan minimization, however, treats jobs as equally important. It captures only the finish time of the last job and therefore cannot distinguish a low-priority order from one whose time in the system is expensive. In order-driven and customized manufacturing, jobs often differ substantially in commercial priority, and weights can reflect costs per unit of time, customer priority tiers, or the urgency of downstream operations. A makespan-optimal schedule may thus postpone exactly those orders for which time in the system is most costly.

Weighting jobs by commercial priority addresses this concern. The weighted completion time w_jC_j assigns each job a completion-time cost: when w_j is a per-unit-time cost, the product gives the job's total delay cost; when w_j encodes urgency (the inverse of customer tolerance), the product expresses system time in urgency-adjusted units. Aggregate formulations typically minimize $\sum w_jC_j$, a problem solved optimally on a single machine by Smith's ratio rule (Smith, 1956). Summation, however, renders one order's poor outcome interchangeable with another's strong one. Weights reflect differences in commercial priority, and such differences do not cancel, so a badly served high-weight order cannot be offset by well-served low-weight orders. A supplier evaluating performance by its worst-served order therefore focuses on the *maximum weighted completion time*, not on the total. These considerations motivate the criterion $\max w_jC_j$, the maximum weighted completion time (also called the *weighted makespan*). Unlike the sum, this criterion identifies the order with the maximum weighted completion time. A job with a large weight can become the bottleneck even when its completion time is not the largest, so the criterion responds to commercial priority rather than to processing time alone. Because it bounds the worst weighted outcome rather than an average, high-priority orders cannot be traded off against low-priority ones.

On a single machine, the largest-weight (LW) rule minimizes the weighted makespan, and processing times are irrelevant (Pinedo, 2022). In a two-machine flow shop, a job's completion time depends on its position in each stage sequence and on the wait time between machines, so processing times matter again and no simple dominance rule determines the order. Apart from makespan, nearly every criterion is $\mathcal{NP}$-hard in this setting (Agnetis et al., 2025). The weighted makespan, a bottleneck objective, retains the weight-based structure that makes

the one-machine case straightforward, yet dependence between stages removes the fixed order on which that structure relies. Its complexity status and practical solvability remain open. Although the weighted makespan has been studied mainly in single-machine settings, its two-machine flow-shop version remains unexplored.

In particular, our model is motivated by a precision components shop that produces custom parts for aerospace, medical, and general industrial customers. Every order follows the same route, CNC machining followed by surface treatment. Each operation handles one order at a time, and no order can begin the second operation before it has finished the first, so the shop is a two-machine flow shop. Weights differ sharply across orders: a surgical implant or an aerospace bracket carries a far larger weight than a standard fastener, because a long completion time on such a part can hold up an operating theatre or an assembly line, whereas a comparable completion time on a fastener only postpones a routine shipment. The shop manager gives priority to high-weight orders and tracks when each order leaves the second operation. Rather than minimizing the total weighted completion time, which lets many low-weight fasteners mask a single implant whose weighted completion time is large, or makespan, which barely changes when one small high-weight order is inserted, the manager focuses on the maximum weighted completion time over all orders.

1.1 Literature review

Research on the maximum weighted completion time objective has primarily focused on single-machine and parallel-machine environments. For single-machine scheduling, approximation algorithms and polynomial-time approximation schemes have been developed in two-agent settings (Nong et al., 2011). Online variants have also received considerable attention, with competitive algorithms established for both preemptive and non-preemptive settings (Li, 2015), followed by online algorithms with best-possible competitive guarantees (Chai et al., 2018). Subsequent studies have extended these settings to incorporate no-delayed-processing constraints (Li and Liu, 2023), unexpected machine breakdowns (Guo et al., 2024), and a single restart of the jobs (Liang et al., 2024). Further research has considered release dates, preemption, and multiple objectives in multi-agent scheduling (Yuan et al., 2020), as well as lot scheduling with the maximum weighted tardiness objective (Mosheiov and Sarig, 2023) and identical parallel machines (Sun, 2024). More recent contributions have addressed order acceptance and rejection decisions (Lu et al., 2025), lot scheduling under the same objective (Zheng et al., 2025), online scheduling on an unbounded parallel-batch machine (Zhang et al., 2025), online settings under (dis)agreeability conditions (Fan et al., 2026), and scheduling with restricted rate- or weight-modifying activities (Zuo et al., 2026a,b). Despite these advances, the maximum weighted completion time objective has received limited attention in flow shop environments, particularly in the two-machine setting. This gap motivates our investigation of the two-machine flow shop scheduling problem with

the objective of minimizing the maximum weighted completion time.

The two-machine flow shop has been extensively studied since Johnson established his rule for makespan minimization (Johnson, 1954). Subsequent research has developed constructive heuristics for various flow shop settings (Shao et al., 2021), while the corresponding three-machine problem remains strongly $\mathcal{NP}$-hard (Pinedo, 2022). Other studies have investigated disruptions (Hnaien and Arbaoui, 2023), branch-and-bound algorithms (Liu and Urgo, 2024a), reentrant production (Liu and Urgo, 2024b), alternative objective functions (Shabtay and Gerstl, 2024), and distributed manufacturing environments (Becker et al., 2025). Meanwhile, refinements of Johnson's rule continue to attract research interest (Della Croce and Schau, 2026).

1.2 Main contributions

The rest of the paper is organized as follows. Section 2 fixes the notation, states the completion-time recurrence and gives the mixed integer programming formulation. Section 3 establishes the complexity of the problem. Section 4 proves the structural properties, develops the exact dynamic program, and presents the approximation schemes. Section 5 describes the 2-approximation and the two heuristics. Section 6 reports the computational study, and Section 7 concludes.

Studying $F2||WC_{\max}$ is worthwhile for three reasons. First, it isolates the effect of weights on the tractability of a bottleneck criterion in serial production: because the unweighted version collapses into the makespan and is solved by Johnson's rule, any difficulty that appears can be attributed to the interaction between weights and two-stage coupling, yielding a clean statement about where the boundary of tractability lies. Second, it tests the transfer of single-machine min–max techniques to flow shops, since the backward greedy construction, the notion of a dominance index, and the role of the last position all must be re-examined once completion times are determined by two sequences instead of one. Third, the criterion is directly usable: industrial scheduling systems are built around makespan and weighted sums, and the sequences they generate carry no guarantee for weighted bottleneck performance; structural properties, bounds, and solution procedures for the weighted makespan supply the missing module and a yardstick against which the loss incurred by makespan-oriented heuristics can be measured.

2 Problem description

Consider a set $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ of n independent and non-preemptive jobs to be scheduled in a two-machine flow shop with machines M_1 and M_2. All jobs are released at time zero and follow the same route, $M_1 \to M_2$. For $j = 1, 2, \ldots, n$, job J_j is represented

by a triple $\Phi_j = (a_j, b_j, w_j)$, where a_j and b_j are its nonnegative integer processing times on M_1 and M_2, respectively, and w_j is a positive weight reflecting its importance. Let K be the number of distinct weights. For a feasible schedule π, let $C_j^1(\pi)$ and $C_j^2(\pi)$ denote the completion times of J_j on M_1 and M_2, respectively; thus $C_j(\pi) = C_j^2(\pi)$ is its completion time. The makespan and the weighted makespan (i.e., the maximum weighted completion time) in π are given by

$$C_{\max}(\pi) = \max_{1\le j\le n}\{C_j(\pi)\} \quad \text{and} \quad C^w_{\max}(\pi) = \max_{1\le j\le n}\{w_j C_j(\pi)\},$$

respectively. The objective is to minimize the weighted makespan $C^w_{\max}$. Using the standard 3-field scheduling notation (Pinedo, 2022), this problem is denoted by $F2 \mid\mid C^w_{\max}$.

In a two-machine flow shop, a schedule is called a *permutation* schedule if the jobs are processed in the same order on both machines. Johnson (1954) showed that an optimal permutation schedule exists for $F2 \mid\mid C_{\max}$. By a similar argument, an optimal permutation schedule also exists for $F2 \mid\mid C^w_{\max}$. Hence, we restrict attention to permutation schedules. Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ denote a feasible permutation schedule for $F2 \mid\mid C^w_{\max}$, where $J_{\pi(j)}$ is the j-th job in π. By simple calculation, it follows that

$$C_{\pi(j)} = \max_{1\le i\le j}\Big\{\sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)}\Big\}, \quad j = 1, 2, \ldots, n. \tag{1}$$

Then, the weighted makespan of π is given by

$$C^w_{\max}(\pi) = \max_{1\le j\le n}\{w_{\pi(j)} C_{\pi(j)}\}. \tag{2}$$

To formulate the problem as a mixed integer linear program (MILP), we introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is the k-th job, and 0 otherwise;
- $C^1_{[k]}$ is the completion time of the k-th job on M_1;
- $C^2_{[k]}$ is the completion time of the k-th job on M_2;
- C_j is the completion time of job J_j;
- M is a sufficiently large constant (e.g., $M = \sum_{j=1}^{n}(a_j + b_j)$).

The MILP model can be formulated as follows:

$$\min \quad C^w_{\max} \tag{3}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, 2, \ldots, n \tag{4}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, 2, \ldots, n \tag{5}$$

$$C^1_{[1]} = \sum_{j=1}^{n} a_j x_{j1} \tag{6}$$

$$C^1_{[k]} = C^1_{[k-1]} + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{7}$$

$$C^2_{[1]} = C^1_{[1]} + \sum_{j=1}^{n} b_j x_{j1} \tag{8}$$

$$C^2_{[k]} \geq C^1_{[k]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{9}$$

$$C^2_{[k]} \geq C^2_{[k-1]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{10}$$

$$C_j \geq C^2_{[k]} - M(1 - x_{jk}), \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{11}$$

$$C^w_{\max} \geq w_j C_j, \quad j = 1, 2, \ldots, n \tag{12}$$

$$x_{jk} \in \{0, 1\}, \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{13}$$

$$C_j, C^1_{[k]}, C^2_{[k]} \geq 0 \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{14}$$

The objective is to minimize $C^w_{\max}$, as expressed in (3). Constraints (4) and (5) assign each job to exactly one position and each position to exactly one job. Constraints (6) and (7) determine the completion times on M_1, and (8) through (10) determine those those on M_2. In particular, for $k \geq 2$, the job in position k can start on M_2 only after it has completed M_1 and the preceding job has finished on M_2. Constraint (11) relates the position-based completion times to the job-based ones via a big-M formulation. Note that when $x_{jk} = 1$ (i.e., J_j is assigned to the k-th position), it requires $C_j \geq C^2_{[k]}$; when $x_{jk} = 0$, it is inactive, since $C^2_{[k]} - M \leq 0$ and $C_j \geq 0$. Constraint (12) imposes $WC_{\max} \geq w_j C_j$ for each job, so minimizing $C^w_{\max}$ yields the optimal objective value of (2). Constraints (13) and (14) require the decision variables x_{jk} to be binary and the completion times nonnegative.

3 Computational complexity

When $K = 1$ (i.e., all jobs have equal weights), $F2 \,||\, C^w_{\max}$ reduces to $F2 \,||\, C_{\max}$, which is solvable in $O(n \log n)$ time (Johnson, 1954). In contrast, We show that $F2 \,||\, C^w_{\max}$ is binary $\mathcal{NP}$-hard even with only two distinct weights and strongly $\mathcal{NP}$-hard when K is arbitrary.

Theorem 3.1. *$F2 \,||\, C^w_{\max}$ is binary $\mathcal{NP}$-hard even with only two distinct weights.*

Proof. To establish the result, we give a polynomial-time reduction from PARTITION, which is $\mathcal{NP}$-complete in the ordinary sense (Garey and Johnson, 1979).

PARTITION: Given r positive integers $x_1, x_2, \ldots, x_r$ and an integer X with $\sum_{i=1}^{r} x_i = 2X$, does there exist a partition of $\{1, 2, \ldots, r\}$ into two disjoint subsets $\mathcal{X}_1$ and $\mathcal{X}_2$ such that $\sum_{i \in \mathcal{X}_1} x_i = \sum_{i \in \mathcal{X}_2} x_i = X$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_r; X)$ of PARTITION, we construct an instance of $F2 \mid\mid C^w_{\max}$ with $n = r + 1$ jobs. For $j = 1, 2, \ldots, n-1$, job J_j has $a_j = 1$, $b_j = rx_j$, and $w_j = X + 1$; job J_n has $a_n = rX$, $b_n = 1$, and $w_n = 2X + 1$. Set the threshold to $Y = r(X+1)(2X+1)$. The question is whether there exists a schedule with $C^w_{\max} \leq Y$.

We demonstrate that $F2 \mid\mid WC_{\max}$ has a feasible schedule π with $C^w_{\max}(\pi) \leq Y$ if and only if PARTITION has a solution.

Suppose that PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2)$ with $\sum_{j \in \mathcal{X}_1} x_j = \sum_{j \in \mathcal{X}_2} x_j = X$. Define $\mathcal{J}_1 = \{J_j : j \in \mathcal{X}_1\}$ and $\mathcal{J}_2 = \{J_{r+j} : j \in \mathcal{X}_2\}$. We build a feasible permutation schedule π in which both machines process the jobs in the order $\mathcal{J}_1 \to J_n \to \mathcal{J}_2$, with the jobs in $\mathcal{J}_1$ and $\mathcal{J}_2$ processed in arbitrary order (see Figure 1).

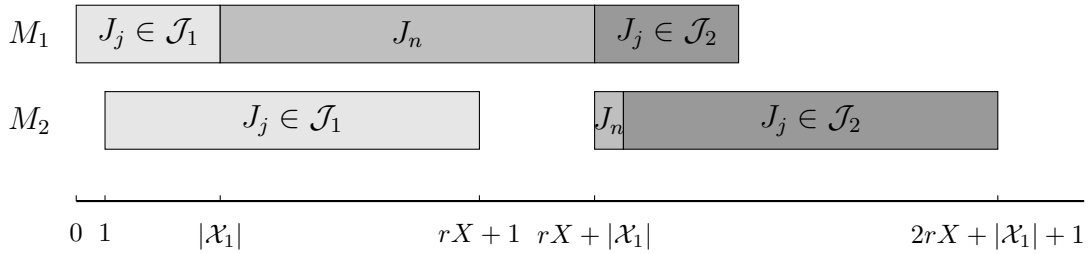


Figure 1: A feasible permutation scheule π

Since $a_j \leq b_j$ for each $j = 1, 2, \ldots, r$ and $|\mathcal{X}_1| \leq r - 1$, it can be easily verified that $C_n(\pi) = rX + |\mathcal{X}_1| + 1 \leq r(X+1)$ and $C_{\max}(\pi) = 2rX + |\mathcal{X}_1| + 1 \leq r(2X+1)$. Hence, $C^w_{\max}(\pi) \leq r(X+1)(2X+1) = Y$.

Conversely, suppose that $F2 \mid\mid C^w_{\max}$ admits a feasible schedule π with $C^w_{\max}(\pi) \leq Y$. Let $\mathcal{J}_1 = \{J_j : C_j(\pi) < C_n(\pi)\}$ and $\mathcal{J}_2 = \{J_j : C_j(\pi) > C_n(\pi)\}$ be the sets of jobs completed before and after J_n in π, respectively, and let $\mathcal{X}_1 = \{j : J_j \in \mathcal{J}_1\}$ and $\mathcal{X}_2 = \{j : J_j \in \mathcal{J}_2\}$. By (1), we have

$$C_n(\pi) \geq \sum_{j \in \mathcal{X}_1} a_j + a_n + 1 = |\mathcal{X}_1| + rX + 1 \tag{15}$$

and

$$C_n(\pi) \geq a_{\pi(1)} + \sum_{j \in \mathcal{X}_1} b_j + b_n = r \sum_{j \in \mathcal{X}_1} x_j + 2. \tag{16}$$

Case 1. If $\sum_{j \in \mathcal{X}_1} x_j > X$, then $\sum_{j \in \mathcal{X}_1} x_j \geq X + 1$ because the x_j are integers. By (16), $WC_{\max}(\pi) \geq w_n C_n(\pi) \geq (2X+1)(r(X+1)+2) > r(X+1)(2X+1) = Y$, contradicting the assumption that $C^w_{\max}(\pi) \leq Y$.

Case 2. If $\sum_{j \in \mathcal{X}_1} x_j < X$, then $\sum_{j \in \mathcal{X}_1} x_j \leq X - 1$ because the x_j are integers. Since

$(\mathcal{X}_1, \mathcal{X}_2)$ is a partition of $\{1, 2, \ldots, r\}$, we have

$$\sum_{j \in \mathcal{X}_2} x_j = 2X - \sum_{j \in \mathcal{X}_1} x_j \geq X + 1. \tag{17}$$

By (1), (15), and (17), we have

$$C_{\pi(n)}(\pi) = C_{\max}(\pi) \geq C_n(\pi) + \sum_{j \in \mathcal{X}_2} b_j \geq |\mathcal{X}_1| + rX + 1 + r \sum_{j \in \mathcal{X}_2} x_j \geq r(2X + 1) + 1. \tag{18}$$

Using $w_{\pi(n)} = 1$, (18) yields

$$C^w_{\max}(\pi) \geq w_{\pi(n)} C_{\pi(n)}(\pi) \geq (X + 1)(r(2X + 1) + 1) > r(X + 1)(2X + 1) = Y,$$

This contradicts the assumption that $C^w_{\max}(\pi) \leq Y$.

Combining the above two cases, we obtain that $\sum_{j \in \mathcal{X}_1} x_j = \sum_{j \in \mathcal{X}_2} x_j = X$. Therefore, $(\mathcal{X}_1, \mathcal{X}_2)$ is a solution to PARTITION. □

Theorem 3.2. *$F2 \,||\, C^w_{\max}$ is strongly $\mathcal{NP}$-hard when K is arbitrary.*

Proof. To establish the result, we give a polynomial-time reduction from 3-PARTITION, which is strongly $\mathcal{NP}$-complete (Garey and Johnson, 1979).

3-PARTITION: Given $3r$ positive integers $x_1, x_2, \ldots, x_{3r}$ and an integer X with $\sum_{i=1}^{3r} x_i = rX$ and $\frac{X}{4} < x_j < \frac{X}{2}$ for each $j = 1, 2, \ldots, 3r$, does there exist a partition of $\{1, 2, \ldots, 3r\}$ into m disjoint subsets $\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r$ such that $\sum_{j \in \mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_{3r}; X)$ of 3-PARTITION, we construct an instance of $F2 \,||\, C^w_{\max}$ consisting of r separating jobs and $3r$ basic jobs. For $j = 1, 2, \ldots, r$, separating job J_j has $a_j = 4X$, $b_j = X$, and $w_j = \frac{5r+4}{5j}$; for $j = r + 1, r + 2, \ldots, 4r$, basic job J_j has $a_j = x_{j-r}$, $b_j = 4x_{j-r}$, and $w_j = 1$. Set the threshold to $Y = (5r + 4)X$. The question is whether there exists a schedule with $WC_{\max} \leq Y$.

We demonstrate that $F2 \,||\, C^w_{\max}$ has a feasible schedule π with $C^w_{\max}(\pi) \leq Y$ if and only if 3-PARTITION has a solution.

Suppose that 3-PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$, so that $\sum_{j \in \mathcal{X}_i} x_j = X$ and $|\mathcal{X}_i| = 3$ for all i. Let $\mathcal{J}_i = \{J_{r+j} : j \in \mathcal{X}_i\}$ for $i = 1, 2, \ldots, r$. We build a feasible permutation schedule π in which both machines process the jobs in the order

$$J_1 \to \mathcal{J}_1 \to J_2 \to \mathcal{J}_2 \to \cdots \to J_r \to \mathcal{J}_r,$$

with the jobs in each $\mathcal{J}_i$ processed in arbitrary order (see Figure 2).

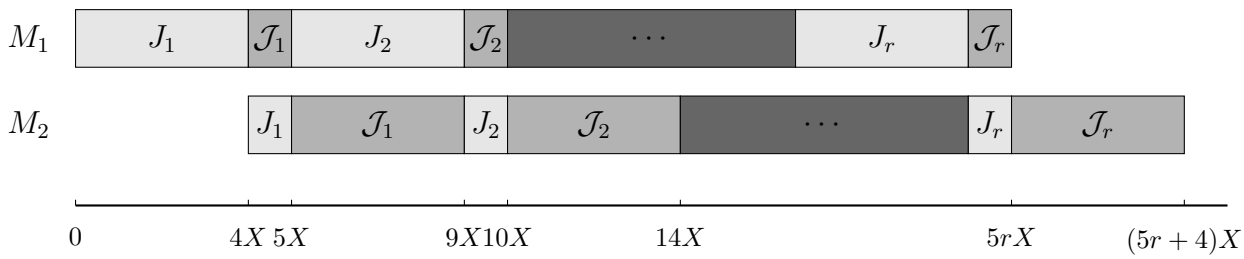


Figure 2: A feasible permutation scheule π

According to Figure 2, it can be easily verified that $C_j(\pi) = 5jX$ for $j = 1, 2, \ldots, r$, and $C_j(\pi) \leq C_{\max}(\pi) = (5r+4)X$ for $j = r+1, r+2, \ldots, 4r$. Then $w_j C_j(\pi) = (5r+4)X = Y$ for $j = 1, 2, \ldots, r$, and $w_j C_j(\pi) \leq w_j C_{\max}(\pi) = (5r+4)X = Y$ for $j = r+1, r+2, \ldots, 4r$. Hence, $C^w_{\max}(\pi) \leq Y$.

Conversely, suppose that $F2 \mid\mid C^w_{\max}$ admits a feasible schedule π with $C^w_{\max}(\pi) \leq Y$. Because all separating jobs have the same processing time on M_1 and M_2 and weights $w_1 > w_2 > \cdots > w_r$, a standard interchange argument lets us assume they appear in π in decreasing order of weight.

Claim 1. $C_{\max}(\pi) \leq Y = (5r+4)X$.

Proof of Claim 1. Since $w_j \geq 1$ for all $1 \leq j \leq 4r$, we have $C_{\max}(\pi) \leq C^w_{\max}(\pi) \leq Y = (5r+4)X$. ■

Claim 2. J_1 is the first job scheduled in π.

Proof of Claim 2. Since $C^w_{\max}(\pi) \leq Y = (5r+4)X$ and $w_1 = \frac{5r+4}{5}$, we have $a_1 + b_1 \leq C_1(\pi) \leq \frac{C^w_{\max}(\pi)}{w_1} \leq 5X = a_1 + b_1$. This means that J_1 is the first job scheduled in π. ■

For $i = 1, 2, \ldots, r-1$, let $\mathcal{B}_i = \{J_j : C_i(\pi) < C_j(\pi) < C_{i+1}(\pi)\}$ be the set of basic jobs scheduled between J_i and J_{i+1} in π, and let $\mathcal{B}_r = \{J_j : C_j(\pi) > C_r(\pi)\}$ be the set of basic jobs scheduled after J_r. For $i = 1, 2, \ldots, r$, define $\mathcal{X}_i = \{j - r : J_j \in \mathcal{B}_i\}$. Then, by Claim 2, π can be written as

$$\pi = (J_1 \to \mathcal{B}_1 \to J_2 \to \mathcal{B}_2 \to \cdots \to J_r \to \mathcal{B}_r). \tag{19}$$

Since $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a partition of $\{1, 2, \ldots, 3r\}$, we have

$$\sum_{i=1}^{r} \sum_{j \in \mathcal{X}_i} x_j = \sum_{j=1}^{3r} x_j = rX. \tag{20}$$

Claim 3. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \leq kX$.

Proof of Claim 3. Suppose to the contrary that there exists some m $(1 \leq m \leq r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j > mX$. Since the x_j are integers, this implies

$$\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \geq mX + 1. \tag{21}$$

Because $a_j = 4X$ and $b_j = X$ for $1 \le j \le r$, and $a_j = x_{j-r}$ for $r+1 \le j \le 4r$, (1), (19), and (21) give

$$C_{m+1}(\pi) \ge \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + b_{m+1} = 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + X \ge 5(m+1)X + 1. \quad (22)$$

Using $w_{m+1} = \frac{5r+4}{5(m+1)}$, (22) yields

$$C^w_{\max}(\pi) \ge w_{m+1} C_{m+1}(\pi) \ge \frac{5r+4}{5(m+1)}(5(m+1)X + 1) > (5r+4)X = Y, \quad (23)$$

which contradicts the assumption that $C^w_{\max}(\pi) \le Y$. Hence, Claim 3 holds. ■

Claim 4. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \ge kX$.

Proof of Claim 4. Suppose to the contrary that there exists some m $(1 \le m \le r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j < mX$. By (20) and the x_j are integers, we have

$$\sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} x_j = rX - \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \ge (r-m)X + 1. \quad (24)$$

Because $a_j = 4X$ and $b_j = X$ for $1 \le j \le r$, and $a_j = x_{j-r}$ and $b_j = 4x_{j-r}$ for $r+1 \le j \le 4r$, (1), (19), (20), and (24) give

$$\begin{aligned} C_{\max}(\pi) &\ge \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + \sum_{j=m+1}^{r} b_j + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{B}_i} b_j \\ &= 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + (r-m)X + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} 4x_j \\ &= 4(m+1)X + (r-m)X + rX + 3 \sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} x_j \\ &\ge 4(m+1)X + (r-m)X + rX + 3((r-m)X + 1) \\ &= (5r+4)X + 3 > (5r+4)X = Y. \end{aligned}$$

which contradicts Claim 1. Hence, Claim 4 holds. ■

Combining Claims 3 and 4 with (20), we obtain that $\sum_{j \in \mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$. Since $X/4 < x_j < X/2$ for each j, a sum equal to X must contain exactly three elements; hence $|\mathcal{X}_i| = 3$ for each i. Therefore, $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a solution to 3-PARTITION. □

4 Fixed number of weights

In this section, we focus on the case where the number of weights K is fixed, i.e., $K = O(1)$. For $K = 1$, the problem is exactly $F2 \,||\, C_{\max}$, which Johnson's rule solves in $O(n \log n)$ time (Johnson, 1954); the rule is summarized in Algorithm 1. For $K = 2$, the problem

$F2 \mid\mid C^w_{\max}$ is already binary $\mathcal{NP}$-hard (Theorem 3.1). For fixed K, we develop an exact DP algorithm running in pseudo-polynomial time and derive an FPTAS from it.

Algorithm 1: Johnson's rule for $F2 \mid\mid C_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j and b_j for $j = 1, 2, \ldots, n$.

1 **Step 1.** Partition $\mathcal{J}$ into two sets $\mathcal{J}^1 = \{J_j : a_j \le b_j\}$ and $\mathcal{J}^2 = \{J_j : a_j > b_j\}$.

2 **Step 2.** Order the jobs in $\mathcal{J}^1$ in nondecreasing order of a_j, followed by those in $\mathcal{J}^2$ in nondecreasing order of b_j.

3 **Step 3.** Process the jobs in this order on M_1 and then on M_2, yielding the permutation schedule π.

Output: The resulting schedule π.

Let $W_1 > W_2 > \cdots > W_K$ denote the distinct weights of the n jobs, and let $\mathcal{G}_k = \{J_j \in \mathcal{J} : w_j = W_k\}$ be set of jobs having weight W_k, $k = 1, 2, \ldots, K$.

4.1 Block structure of an optimal schedule

To establish the key structural lemma, we first introduce some notation for partitioning a permutation schedule into blocks.

Definition 4.1. ***(Block partition)*** *Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ be a permutation schedule. Set $\ell_0 = 0$ and $\mathcal{R}_1 = \mathcal{J}$. For $k = 1, 2, \ldots, K$, if $\mathcal{R}_k \cap \mathcal{G}_k \neq \emptyset$, define $\ell_k = \max\{j : J_{\pi(j)} \in \mathcal{R}_k \cap \mathcal{G}_k\}$, $\mathcal{B}^\pi_k = \{J_{\pi(\ell_{k-1}+1)}, J_{\pi(\ell_{k-1}+2)}, \ldots, J_{\pi(\ell_k)}\}$, and $\mathcal{R}_{k+1} = \mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}^\pi_i$. Otherwise, set $\ell_k = \ell_{k-1}$, $\mathcal{B}^\pi_k = \emptyset$, and $\mathcal{R}_{k+1} = \mathcal{R}_k$. Then we call $\mathcal{B}(\pi) = (\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K)$ the block partition of π.*

Thus, ℓ_k is the position of the last job in $\mathcal{R}_k$ of weight W_k, $\mathcal{B}^\pi_k$ is the k-th block (possibly empty), and $\mathcal{R}_k$ is the set of jobs left after removing the first $k-1$ blocks. By construction, every nonempty block $\mathcal{B}^\pi_k$ ends with a job of weight W_k (i.e., $w_{\pi(\ell_k)} = W_k$), and all jobs in the block have weight at most W_k, i.e., $w_{\pi(j)} \le W_k$ for all $\ell_{k-1} + 1 \le j \le \ell_k$. This means that

$$W_k C_{\pi(\ell_k)}(\pi) = \max\{w_{\pi(j)} C_{\pi(j)}(\pi) : \ell_{k-1} + 1 \le j \le \ell_k\}. \tag{25}$$

Consequently, (2) can be reformulated as

$$C^w_{\max}(\pi) = \max_{1 \le k \le K} \{W_k C_{\pi(\ell_k)(\pi)}\}. \tag{26}$$

Definition 4.2. ***(Regular block and regular permutation schedule)*** *Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ be a permutation schedule with block partition $\mathcal{B}(\pi) = (\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K)$. A block $\mathcal{B}^\pi_k$ is called regular if it is empty or if the sequence of its jobs in π matches the sequence obtained by applying Johnson's rule to that block. The schedule π is called regular if all blocks $\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K$ are regular.*

To illustrate the block partition, we give a small numerical example for $F2 \mid\mid C^w_{\max}$.

Example 4.1. *Consider a six-job instance of* $F2 \,||\, C^w_{\max}$ *whose job data are given in Table 1, and let* $\pi = (1, 2, \ldots, 6)$ *be a permutation schedule.*

Table 1: Job data for Example 4.1

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	2	3
b_j	5	3	9	4	3	3
w_j	2	3	5	2	3	2

In Example 4.1, we have $K = 3$, $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_2, J_5\}$, and $\mathcal{G}_3 = \{J_1, J_4, J_6\}$. The resulting blocks are $\mathcal{B}^\pi_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}^\pi_2 = \{J_4, J_5\}$, and $\mathcal{B}^\pi_3 = \{J_6\}$ (see Figure 3). By (26), the weighted makespan of π is $C^w_{\max}(\pi) = \max\{5 \times 22, 3 \times 29, 2 \times 32\} = 110$.

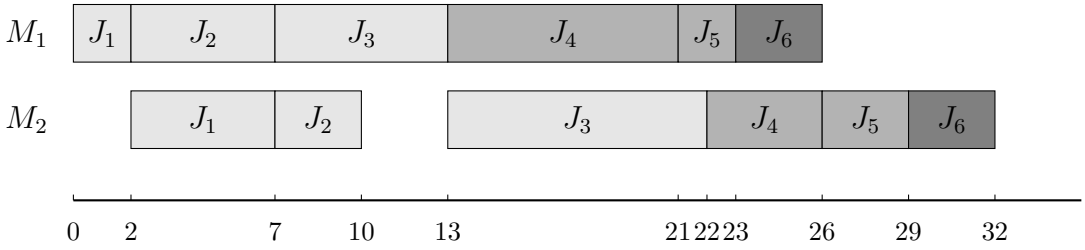


Figure 3: A block partition of π

Given a permutation schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$, for $1 \le g \le h \le n$, define

$$C^w_{\max}(\pi)[g : h] = \max_{g \le j \le h} \{w_{\pi(j)} C_{\pi(j)}(\pi)\}$$

as the maximum weighted completion time over the jobs $J_{\pi(g)}, J_{\pi(g+1)}, \ldots, J_{\pi(h)}$.

The following lemma shows that if the jobs in each block of π are rescheduled in accordance with Johnson's rule, the resulting permutation schedule π' has objective value no larger than that of π, i.e., $C^w_{\max}(\pi') \le C^w_{\max}(\pi)$.

Lemma 4.1. *Given a permutation schedule* π *and its block partition* $\mathcal{B}(\pi) = (\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K)$, *let* π_k *be the permutation of* $\mathcal{B}^\pi_k$ *obtained by Johnson's rule for* $k = 1, 2, \ldots, K$. *Then the schedule* π' *formed by concatenating* $\pi_1, \pi_2, \ldots, \pi_K$ *satisfies* $C^w_{\max}(\pi') \le C^w_{\max}(\pi)$.

Proof. First, if $\mathcal{B}^\pi_k \ne \emptyset$, then rescheduling the jobs in $\mathcal{B}^\pi_k$ according to Johnson's rule does not change the set of jobs assigned to positions $\ell_{k-1} + 1, \ell_{k-1} + 2, \ldots, \ell_k$, i.e.,

$$\mathcal{B}^\pi_k = \{J_{\pi(\ell_{k-1}+1)}, J_{\pi(\ell_{k-1}+2)}, \ldots, J_{\pi(\ell_k)}\} = \{J_{\pi'(\ell_{k-1}+1)}, J_{\pi'(\ell_{k-1}+2)}, \ldots, J_{\pi'(\ell_k)}\}. \tag{27}$$

Thus, in π', the jobs in $\mathcal{B}^\pi_k$ still occupy this same index interval. By (25), we have

$$C^w_{\max}(\pi)[\ell_{k-1} + 1 : \ell_k] = W_k C_{\pi(\ell_k)}(\pi) = w_{\pi(\ell_k)} C_{\pi(\ell_k)}(\pi), k = 1, 2, \ldots, K. \tag{28}$$

Moreover, by (27), for $k = 1, 2, \ldots, K$, $\mathcal{B}_k^\pi$ has the same completion time on M_1 under π and π', i.e., $C^1_{\pi'(\ell_k)}(\pi') = C^1_{\pi(\ell_k)}(\pi) = \sum_{j=1}^{\ell_k} a_{\pi(j)}$.

Second, we prove by induction on k that, for all $k = 1, 2, \ldots, K$,

$$C_{\pi'(\ell_k)}(\pi') \le C_{\pi(\ell_k)}(\pi) \text{ and } C^w_{\max}(\pi')[\ell_{k-1}+1 : \ell_k] \le C^w_{\max}(\pi)[\ell_{k-1}+1 : \ell_k]. \tag{29}$$

For the base case $i = 1$, both machines are available at time zero. Rescheduling the jobs in $\mathcal{B}_1^\pi$ according to Johnson's rule gives $C_{\pi'(\ell_1)}(\pi') \le C_{\pi(\ell_1)}(\pi)$. Since π' only permutes the jobs in $\mathcal{B}_1^\pi$, we have $w_{\pi'(j)} \le W_1$ for $j = 1, 2, \ldots, \ell_1$. Hence, we have

$$C^w_{\max}(\pi')[1 : \ell_1] = \max_{1 \le j \le \ell_1} \{w_{\pi'(j)} C_{\pi'(j)}(\pi')\} \le W_1 C_{\pi'(\ell_1)}(\pi') \le W_1 C_{\pi(\ell_1)}(\pi) = C^w_{\max}(\pi)[1 : \ell_1],$$

where the last equality follows from (28)

Assume that (29) holds for all $i = 1, 2, \ldots, k$. We now prove the $(k+1)$-block. If $\mathcal{B}_{k+1}^\pi = \emptyset$, the assertion is immediate. It remains to consider $\mathcal{B}_{k+1}^\pi \neq \emptyset$. In this case, in π', by the inductive hypothesis, M_1 becomes available at time $C^1_{\pi'(\ell_k)}(\pi') = C^1_{\pi(\ell_k)}(\pi)$, and M_2 is available at time $C_{\pi'(\ell_k)}(\pi') \le C_{\pi(\ell_k)}(\pi)$. We introduce a dummy job J_0 with processing times $a_0 = 0$ and $b_0 = C_{\pi'(\ell_k)}(\pi') - C^1_{\pi'(\ell_k)}(\pi')$, and apply Johnson's rule to schedule the jobs in $\mathcal{B}_{k+1}^\pi \cup \{J_0\}$, with both machines available from time $C^1_{\pi'(\ell_k)}(\pi')$. Because $a_0 = 0$, J_0 is scheduled in the first position according to Johnson's rule, and it completes at time $C_{\pi'(\ell_k)}(\pi')$. By positioning the dummy job in this way, the jobs in $\mathcal{B}_{k+1}^\pi$ can be scheduled on M_2 without overlapping with the previously scheduled jobs in $\mathcal{B}_1^\pi \cup \mathcal{B}_2^\pi \cup \cdots \cup \mathcal{B}_k^\pi$. Since the jobs in $\mathcal{B}_{k+1}$ are scheduled according to Johnson's rule, it follows that $C_{\pi'(\ell_{k+1})}(\pi') \le C_{\pi(\ell_{k+1})}(\pi)$. Moreover, since π' only permutes the jobs in $\mathcal{B}_{k+1}^\pi$, we have $w_{\pi'(j)} \le W_{k+1}$ for $j = \ell_k + 1, \ell_k + 2, \ldots, \ell_{k+1}$. Hence,

$$\begin{aligned} C^w_{\max}(\pi')[\ell_k + 1 : \ell_{k+1}] &= \max_{\ell_k+1 \le j \le \ell_{k+1}} \{w_{\pi'(j)} C_{\pi'(j)}(\pi')\} \le W_{k+1} C_{\pi'(\ell_{k+1})}(\pi') \\ &\le W_{k+1} C_{\pi(\ell_{k+1})}(\pi) = C^w_{\max}(\pi)[\ell_k + 1 : \ell_{k+1}], \end{aligned}$$

where the last equality follows from (28). This completes the inductive step.

Note that $C^w_{\max}(\pi') = \max_{1 \le k \le K} \{C^w_{\max}(\pi')[\ell_{k-1}+1 : \ell_k]\}$ and $C^w_{\max}(\pi) = \max_{1 \le k \le K} \{C^w_{\max}(\pi)[\ell_{k-1}+1 : \ell_k]\}$. Hence, (29) yields $C^w_{\max}(\pi') \le C^w_{\max}(\pi)$. □

To illustrate Lemma 4.1, consider Example 4.1. Given π and its block partition $\mathcal{B}(\pi) = (\mathcal{B}_1^\pi, \mathcal{B}_2^\pi, \mathcal{B}_3^\pi)$, we apply Johnson's rule independently within each block $\mathcal{B}_k^\pi$, $k = 1, 2, 3$, obtaining a permutation π_k of the jobs in that block. Concatenating π_1, π_2, π_3 gives $\pi' = (1, 3, 2, 5, 4, 6)$, with blocks $\mathcal{B}_1^{\pi'} = \{J_1, J_3\}$, $\mathcal{B}_2^{\pi'} = \{J_2, J_5\}$, and $\mathcal{B}_3^{\pi'} = \{J_4, J_6\}$ (see Figure 4). By (26), the weighted makespan of π' is $C^w_{\max}(\pi') = \max\{5 \times 17, 3 \times 23, 2 \times 30\} = 85$.

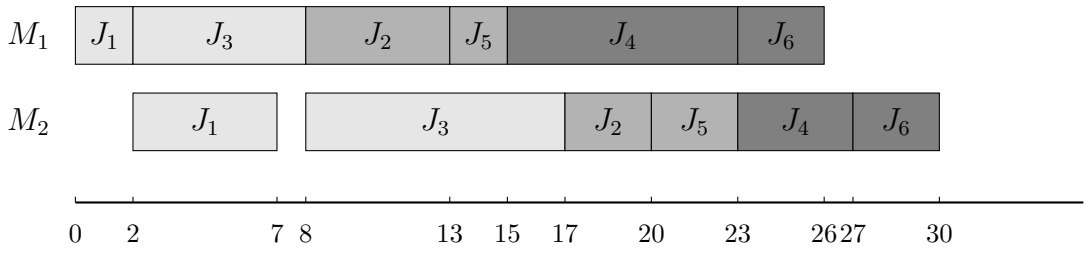


Figure 4: An applicaiton of Lemma 4.1 to π

Lemma 4.2. *For $F2 \mid\mid C^w_{\max}$, there exists an optimal permutation schedule that is regular.*

Proof. Let π be an optimal permutation schedule for $F2 \mid\mid C^w_{\max}$. If π is regular, the lemma follows. Otherwise, let $\mathcal{B}(\pi) = (\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K)$ be its block partition. Apply Johnson's rule within each block $\mathcal{B}^\pi_k$ $(k = 1, 2, \ldots, K)$ to obtain a permutation π_k, and concatenate $\pi_1, \pi_2, \ldots, \pi_K$ to form a permutation schedule π'. By Lemma 4.1, $C^w_{\max}(\pi') \leq C^w_{\max}(\pi)$. Since π is optimal, π' is optimal as well.

If π' is regular, the lemma follows. Otherwise, repeat the above procedure with π' in place of π. For termination, suppose the first i blocks of the current schedule are already regular (initially $i = 0$). By definition, applying Johnson's rule to a regular block leaves its job sequence unchanged. Hence, the first i blocks remain unchanged. The $(i+1)$-th block in the resulting schedule may contain a different job set, but it satisfies Johnson's rule. Thus the first $i+1$ blocks are regular, so every application increases the number of leading regular blocks by at least one. Since there are at most K nonempty blocks in each schedule, the procedure terminates after at most K iterations and yields an optimal regular schedule. □

Let π be a regular permutation schedule for $F2 \mid\mid C^w_{\max}$ with block partition $\mathcal{B}(\pi) = (\mathcal{B}^\pi_1, \mathcal{B}^\pi_2, \ldots, \mathcal{B}^\pi_K)$. For each $k = 1, 2, \ldots, K$, let π_k denote the schedule obtained by processing the jobs in $\mathcal{B}^\pi_k$ consecutively from time zero in accordance with Johnson's rule. Note that π coincides with the schedule obtained by concatenating $\pi_1, \pi_2, \ldots, \pi_K$. This representation allows us to compute $C_{\max}(\pi)$ and $C^w_{\max}(\pi)$ from parameters defined for each π_k. Specifically, for each k, let $A_k = A(\pi_k)$ and $B_k = B(\pi_k)$ denote the total processing times of the jobs in π_k on M_1 and M_2, respectively, and let $C_k = C(\pi_k)$ denote the makespan of π_k. Given these parameters, we describe an procedure for computing $C_{\max}(\pi)$ and $C^w_{\max}(\pi)$. The procedure starts with π_1 and successively appends $\pi_2, \pi_3, \ldots, \pi_K$ in that order. For $k = 1, 2, \ldots, K$, let $\pi[1:k]$ denote the schedule obtained by concatenating $\pi_1, \pi_2, \ldots, \pi_k$; thus $\pi = \pi[1:K]$. The makespan and weighted makespan of $\pi[1:k]$ are computed recursively. For convenience, set $C_{\max}(\pi[1:0]) = C^w_{\max}(\pi[1:0]) = 0$. For $k = 1, 2, \ldots, K$, by direct calculation and induction

on k, we have

$$\begin{aligned} C_{\max}(\pi[1:k]) &= \max\Big\{\sum_{i=1}^{k-1} A_i + C_k, C_{\max}(\pi[1:k-1]) + B_k\Big\} \\ &= \max_{0\le h\le k-1}\Big\{\sum_{i=1}^{h} A_i + C_{h+1} + \sum_{i=h+2}^{k} B_i\Big\} \end{aligned} \tag{30}$$

and

$$\begin{aligned} C^w_{\max}(\pi[1:k]) &= \max\{C^w_{\max}(\pi[1:k-1]), W_k C_{\max}(\pi[1:k])\} \\ &= \max_{1\le h\le k}\{W_h C_{\max}(\pi[1:h])\}. \end{aligned} \tag{31}$$

Hence, $C^w_{\max}(\pi) = C^w_{\max}(\pi[1:K])$.

4.2 An exact DP algorithm

Recall that $W_1 > W_2 > \cdots W_K$. For each job J_j $(j = 1, 2, \ldots, n)$, let $g_j \in \{1, 2, \ldots, K\}$ be the index such that $w_j = W_{g_j}$, i.e., $J_j \in \mathcal{G}_{g_j}$. Assume that the jobs in $\mathcal{J}$ are indexed in accordance with Johnson's rule.

Building on the preceding discussion in Section 4.1, we develop an exact DP algorithm for $F2 \mid\mid C^w_{\max}$. It generates a sequence of sets $\mathcal{H}_0, \mathcal{H}_1, \ldots, \mathcal{H}_n$. Each $\mathcal{H}_j$ consists of K-tuples $(X_1, X_2, \ldots, X_K)$, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$. A tuple in $\mathcal{H}_j$ $(j = 1, 2, \ldots, n)$ encodes an assignment of jobs $J_1, J_2, \ldots, J_j$ to K schedules (i.e., blocks) $\pi_1(j), \pi_2(j), \ldots, \pi_K(j)$, with jobs in each schedule processed from time zero in increasing order of their indices. For each schedule $\pi_k(j)$, A_k and B_k are the total processing times on M_1 and M_2, respectively, and C_k and L_k are the makespan of $\pi_k(j)$ and the weight of its last job.

Initially, $\mathcal{H}_0$ consists of a single tuple, all of whose $4K$ entries are zero, i.e., $\mathcal{H}_0 = \{\big(\underbrace{(0,0,0,0),\ldots,(0,0,0,0)}_{K \text{ copies}}\big)\}$. In the j-th iteration $(j = 1, 2, \ldots, n)$, $\mathcal{H}_j$ is constructed from $\mathcal{H}_{j-1}$. For each tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_{j-1}$, with $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$, the definition of g_j and Lemma 4.2 together imply that this tuple generates g_j new tuples in $\mathcal{H}_j$. If $k \le g_j$, appending J_j to $\pi_k(j-1)$ yields the new tuple $(X_1, \ldots, X_{k-1}, X'_k, X_{k+1}, \ldots, X_K)$ into $\mathcal{H}_j$, where $X'_k = (A'_k, B'_k, C'_k, L'_k)$ is given by

$$A'_k = A_k + a_j,\ B'_k = B_k + b_j,\ C'_k = \max\{C_k, A_k + a_j\} + b_j,\ L'_k = w_j. \tag{32}$$

The following lemma will be used to remove dominated tuples during the tuple generation, which can be easily verified.

Lemma 4.3. *Let $X = (X_1, X_2, \ldots, X_K)$ and $X' = (X'_1, X'_2, \ldots, X'_K)$ be two tuples in $\mathcal{H}_j$, with $X_k = (A_k, B_k, C_k, L_k)$ and $X'_k = (A'_k, B'_k, C'_k, L'_k)$ for $k = 1, 2, \ldots, K$. If*

$$A_k = A'_k,\ B_k = B'_k,\ C_k = C'_k,\ L_k = L'_k\ (k = 1, 2, \ldots, K-1),$$

$L_K = L'_K$, and $C_K \leq C'_K$, then X' is dominated by X and can therefore be deleted from $\mathcal{H}_j$.

By Lemma 4.2, it suffices to consider regular permutation schedules. Therefore, once $\mathcal{H}_n$ has been generated, we discard every tuple $(X_1, X_2, \ldots, X_K)$ with $L_k \notin \{0, W_k\}$ for some k, since concatenating the schedules corresponding to $X_1, X_2, \ldots, X_K$ cannot yield a regular schedule. This does not affect our search for the optimal solution. Then, for each retained tuple in $\mathcal{H}_n$, we compute the makespan and weighted makespan according to (30) and (31), respectively.

Summarizing the above analysis and utilizing Lemma 4.2 and Lemma 4.3, the following algorithm is developed to solve $F2 \mid\mid C^w_{\max}$.

Algorithm 2: An exact DP algorithm for $F2 \mid\mid C^w_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j, b_j, w_j, and g_j for $j = 1, 2, \ldots, n$, K distinct weights with $W_1 > W_2 > \cdots > W_K$.

Step 1. [Preprocessing] Renumber jobs in $\mathcal{J}$ according to Johnson' rule (Algorithm 1).

Step 2. [Initialization] Set $\mathcal{H}_0 = \{\big(\underbrace{(0,0,0,0),\ldots,(0,0,0,0)}_{K \text{ copies}}\big)\}$, and $\mathcal{H}_j = \emptyset$ for $j = 1, 2, \ldots, n$.

Step 3. [Generation of sets $\mathcal{H}_1, \mathcal{H}_2, \ldots, \mathcal{H}_n$]

for $j = 1$ **to** n **do**

  for *each* $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_{j-1}$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

    for $k = 1$ **to** g_j **do**

      $A'_k = A_k + a_j$, $B'_k = B_k + b_j$, $C'_k = \max\{C_{k-1}, A_{k-1} + a_j\} + b_j$, $L'_k = w_j$.

      $X'_k = (A'_k, B'_k, C'_k, L'_k)$.

      Add the tuple $(X_1, \ldots, X_{k-1}, X'_k, X_{k+1}, \ldots, X_K)$ to $\mathcal{H}_j$.

    end

  end

  [Elimination] Among all tuples in $\mathcal{H}_j$ with the same $X_1, X_2, \ldots, X_{K-1}$ and the same L_K, retain only the tuple with the smallest C_K (by Lemma 4.3).

end

Step 4. [Computation of objective values for the tuples in $\mathcal{H}_n$]

for *each* $X = (X_1, X_2, \ldots, X_K) \in \mathcal{H}_n$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

  if $L_k \in \{0, W_k\}$ *for each* $k = 1, 2, \ldots, K$ **then**

    Set $T^1_0 = 0$, $T^2_0 = 0$, $Z_0 = 0$.

    for $k = 1$ **to** K **do**

      $T^1_k = T^1_{k-1} + A_k$, $T^2_k = \max\{T^1_{k-1} + C_k,\ T^2_{k-1} + B_k\}$.

      $Z_k = \max\{Z_{k-1}, L_k \cdot T^2_k\}$.

    end

    $C^w_{\max}(X) = Z_K$.

  else

    $C^w_{\max}(X) = +\infty$.

  end

end

Step 5. [Result] Compute $C^w_{\max}(X_{\text{opt}}) = \min\{C^w_{\max}(X) : X \in \mathcal{H}_n\}$ and recover the corresponding optimal schedule π_{opt} from X_{opt} by backtracking.

Output: The optimal schedule π_{opt}.

Theorem 4.1. *Algorithm 2 solves* $F2 \mid\mid C^w_{\max}$ *in* $O(nK(K+1)!2^{K-1}P^{3K-3})$ *time, where* $P = \max\{\sum_{j=1}^n a_j, \sum_{j=1}^n b_j\}$. *For fixed* K, *it runs in* $O(nP^{3K-3})$ *time and is therefore pseudo-polynomial.*

Proof. The correctness follows from three facts: (i) the algorithm enumerates all tuples corresponding to regular schedules, as characterized by Definition 4.2 and Lemma 4.2; (ii)

the elimination operation deletes only dominated tuples, as guaranteed by Lemma 4.3; and (iii) the weighted makespan of a regular schedule is computed using (31).

Step 1 sorts in $O(n \log n)$ time, and Step 2 initializes in $O(n)$ time. For each $j = 1, 2, \ldots, n$ and each tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j$, write $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$. The components satisfy $A_k \le \sum_{i=1}^{j} a_i \le P$, $B_k \le \sum_{i=1}^{j} b_i \le P$, $C_k \le \sum_{i=1}^{j} a_i + \sum_{i=1}^{j} b_i \le 2P$, $L_k \in \{0, W_k, W_{k+1}, \ldots, W_K\}$ for $k = 1, 2, \ldots, K$. Because $\sum_{k=1}^{K} A_k = \sum_{i=1}^{j} a_i$ and $\sum_{k=1}^{K} B_k = \sum_{i=1}^{j} b_i$, only $K-1$ of the A_k and $K-1$ of the B_k are independent. After elimination, the number of distinct tuples in $\mathcal{H}_j$ is at most $O((K+1)!P^{2K-2}(2P)^{K-1}) = O((K+1)!2^{K-1}P^{3K-3})$. In addition, each tuple in $\mathcal{H}_{j-1}$ generates $g_j \le K$ new tuples in $\mathcal{H}_j$. Thus, $\mathcal{H}_j$ can be generated in $O(K(K+1)!2^{K-1}P^{3K-3})$ time. Summing the cost of Step 3 over its n iterations gives $O(nK(K+1)!2^{K-1}P^{3K-3})$ time. Step 4 computes the objective value of each tuple in $\mathcal{H}_n$ in $O(K)$ time and thus requires $O(K(K+1)!2^{K-1}P^{3K-3})$ time. Step 5 requires $O((K+1)!2^{K-1}P^{3K-3})$ time. Consequently, the algorithm runs in $O(nK(K+1)!2^{K-1}P^{3K-3})$ time. □

Applying Algorithm 2 to Example 4.1, we obtain an optimal schedule $\pi^* = (3, 5, 6, 2, 1, 4)$, with blocks $\mathcal{B}_1^{\pi^*} = \{J_3\}$, $\mathcal{B}_2^{\pi^*} = \{J_5, J_6, J_2\}$, and $\mathcal{B}_3^{\pi^*} = \{J_1, J_4\}$ (see Figure 5). The weighted makespan of π^* is $C_{\max}^{w}(\pi^*) = \max\{5 \times 15, 3 \times 24, 2 \times 33\} = 75$.

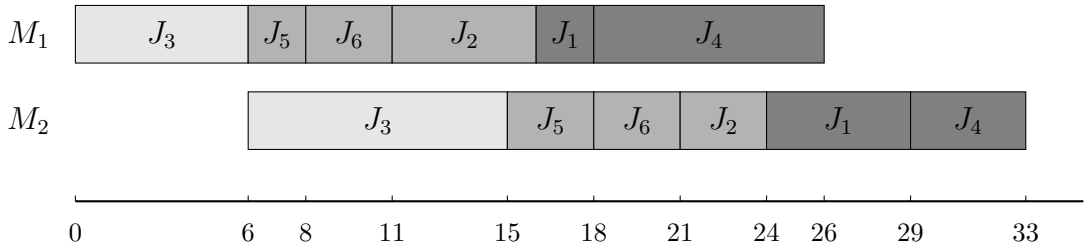


Figure 5: An optimal scheule π^*

4.3 An FPTAS for fixed number of weights

Let $0 < \epsilon \le 1$ be real, and let $\delta = 1 + \epsilon/(2n)$. Define the function $\Delta(\cdot)$ on $[0, +\infty)$ by

$$\Delta(z) = \lfloor \log_\delta z \rfloor \quad (z \ge 1), \qquad \Delta(0) = -1. \tag{33}$$

Equivalently, if k is an integer that satisfies $\delta^k \le z < \delta^{k+1}$, then $\Delta(z) = k$.

For fixed K, we apply the geometric rounding technique to transform Algorithm 2 into an FPTAS (which we call Algorithm 3). In contrast to Algorithm 2, the proposed algorithm may remove some non-dominated states in the tuple set generation procedure. Let $\mathcal{H}_j^{\epsilon}$ be the set of tuples generated by Algorithm 3 in the j-th iteration. For each tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j^{\epsilon}$, define its label to be $(Y_1, Y_2, \ldots, Y_K)$, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$; the first $K-1$ labels are

$$Y_k = (\Delta(A_k), \Delta(B_k), \Delta(C_k), L_k), \quad 1 \le k \le K-1,$$

while the last label is

$$Y_K = (\Delta(A_K), \Delta(B_K), L_K).$$

By construction, for each $1 \le k \le K$ with $A_k \ge 1$, the rounded value $\delta^{\Delta(A_k)}$ approximates A_k up to a small multiplicative error; likewise, $\delta^{\Delta(B_k)}$ and $\delta^{\Delta(C_k)}$ approximate B_k and C_k, respectively. Hence, whenever two tuples share a label, one of them is eliminated during tuple elimination procedure. A formal description of Algorithm 3 is given below.

Algorithm 3: An FPTAS for $F2 \,||\, C^w_{\max}$

Input: Job set $\mathcal{J}$; error parameter ϵ; job parameters a_j, b_j, w_j, and g_j for $j = 1, 2, \ldots, n$; K distinct weights with $W_1 > W_2 > \cdots > W_K$.

Step 1. [Preprocessing] Renumber jobs in $\mathcal{J}$ according to Johnson' rule (Algorithm 1).

Step 2. [Initialization] Set $\delta = 1 + \epsilon/(2n)$. Let $S_0 = \Big(\underbrace{(0,0,0,0),\ldots,(0,0,0,0)}_{K \text{ copies}}\Big)$, and let $R_0 = \Big(\underbrace{(-1,-1,-1,0),\ldots,(-1,-1,-1,0)}_{K-1 \text{ copies}},(-1,-1,0)\Big)$ be its label. Initialize $\mathcal{H}^\epsilon_0 = \{S_0\}$ and $\mathcal{H}^\epsilon_j = \emptyset$ for $j = 1, 2, \ldots, n$.

Step 3. [Generation of sets $\mathcal{H}^\epsilon_1, \mathcal{H}^\epsilon_2, \ldots, \mathcal{H}^\epsilon_n$]

for $j = 1$ **to** n **do**

  for *each* $(X_1, X_2, \ldots, X_K) \in \mathcal{H}^\epsilon_{j-1}$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

    for $k = 1$ **to** g_j **do**

      $A'_k = A_k + a_j$, $B'_k = B_k + b_j$, $C'_k = \max\{C_{k-1}, A_{k-1} + a_j\} + b_j$, $L'_k = w_j$.

      $X'_k = (A'_k, B'_k, C'_k, L'_k)$.

      if $k \le K - 1$ **then**

        $Y'_k = (\Delta(A'_k), \Delta(B'_k), \Delta(C'_k), L'_k)$.

      else

        $Y'_K = (\Delta(A'_K), \Delta(B'_K), L'_K)$.

      end

      Add the tuple $(X_1, \ldots, X_{k-1}, X'_k, X_{k+1}, \ldots, X_K)$ to $\mathcal{H}^\epsilon_j$ and attach the associated label $(Y_1, \ldots, Y_{k-1}, Y'_k, Y_{k+1}, \ldots, Y_K)$ to it.

    end

  end

  [Elimination] Among tuples in $\mathcal{H}^\epsilon_j$ sharing identical labels, retain only the one with the smallest C_K.

end

Step 4. [Computation of objective values for the tuples in $\mathcal{H}^\epsilon_n$]

for *each* $X = (X_1, X_2, \ldots, X_K) \in \mathcal{H}^\epsilon_n$ *with* $X_k = (A_k, B_k, C_k, L_k)$, $k = 1, 2, \ldots, K$ **do**

  Perform the same operations as in Step 4 of Algorithm 2.

end

Step 5. [Result] Compute $C^w_{\max}(X') = \min\{C^w_{\max}(X) : X \in \mathcal{H}^\epsilon_n\}$ and recover the corresponding approximate schedule π' from X' by backtracking.

Output: The approximate schedule π'.

In Algorithm 3, the box-elimination operation considers coordinate values that increase

geometrically. Multiplicative errors can occur if, during tuple elimination, each label is represented by the smallest C_X coordinates inside its box rather than by its exact value. Nevertheless, the lemma below demonstrates that the tuples left after elimination form a good approximation to those produced by Algorithm 2.

Lemma 4.4. *Given any tuple $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j$ generated by Algorithm 2, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$, Algorithm 3 returns a tuple $(\widetilde{X_1}, \widetilde{X_2}, \ldots, \widetilde{X_K}) \in \mathcal{H}_j^\epsilon$, with $\widetilde{X_k} = (\widetilde{A_k}, \widetilde{B_k}, \widetilde{C_k}, L_k)$, such that*

$$\widetilde{A_k} \le \delta^j A_k, \quad \widetilde{B_k} \le \delta^j B_k, \quad \widetilde{C_k} \le \delta^j C_k, \quad k = 1, 2, \ldots, K. \tag{34}$$

Proof. We prove the result by induction on $j = 1, 2, \ldots, n$. For the base case $j = 1$, J_1 can be assigned to each of the g_1 schedules (each induced by a block) in Algorithm 2 and Algorithm 3. Since these g_1 tuples receive distinct labels in Algorithm 3, it follows that $\mathcal{H}_1^\epsilon = \mathcal{H}_1$, so (34) holds for $j = 1$.

Assume that (34) holds for $j - 1$ (the induction hypothesis), where $j \ge 2$. Let $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j$ be a tuple obtained from Algorithm 2 for the first j jobs, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$. When implementing Algorithm 2, the tuple $(X_1, X_2, \ldots, X_K)$ is constructed from a tuple $(X'_1, X'_2, \ldots, X'_K) \in \mathcal{H}_{j-1}$, where $X'_k = (A'_k, B'_k, C'_k, L'_k)$ for $k = 1, 2, \ldots, K$. Specifically, J_j is assigned to one of the g_j schedules corresponding to $(X'_1, X'_2, \ldots, X'_K)$ for the first $j-1$ jobs. Based on the induction hypothesis, there exists a tuple $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_K}) \in \mathcal{H}_{j-1}^\epsilon$, where $\widetilde{X'_k} = (\widetilde{A'_k}, \widetilde{B'_k}, \widetilde{C'_k}, L'_k)$, such that

$$\widetilde{A'_k} \le \delta^{j-1} A'_k, \quad \widetilde{B'_k} \le \delta^{j-1} B'_k, \quad \widetilde{C'_k} \le \delta^{j-1} C'_k, \quad k = 1, 2, \ldots, K. \tag{35}$$

To prove that (34) holds for j, consider the g_j possible cases in which J_j is assigned to one of these schedules.

Let J_j be assigned to the h-th schedule in Algorithm 2 for some $h \in \{1, 2, \ldots, g_j\}$. Then

$$\begin{aligned} &A_k = A'_k, \quad B_k = B'_k \ , C_k = C'_k \ , L_k = L'_k, \quad k \in \{1, 2, \ldots, K\} \setminus \{h\}; \\ &A_h = A'_h + a_j, \quad B_h = B'_h + b_j, \quad C_h = \max\{C'_h, A'_h + a_j\} + b_j, \quad L_h = w_j. \end{aligned} \tag{36}$$

Since $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_K}) \in \mathcal{H}_{j-1}^\epsilon$, the tuple $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_{h-1}}, \overline{X'_h}, \widetilde{X'_{h+1}}, \ldots, \widetilde{X'_K})$ is added to $\mathcal{H}_j^\epsilon$ in Algorithm 3, where $\overline{X'_h} = (\overline{A'_h}, \overline{B'_h}, \overline{C'_h}, \overline{L'_h})$ is defined by

$$\overline{A'_h} = \widetilde{A'_h} + a_j, \quad \overline{B'_h} = \widetilde{B'_h} + b_j, \quad \overline{C'_h} = \max\{\widetilde{C'_h}, \widetilde{A'_h} + a_j\} + b_j, \quad \overline{L'_h} = w_j. \tag{37}$$

Because two tuples can share a label, $(\widetilde{X'_1}, \widetilde{X'_2}, \ldots, \widetilde{X'_{h-1}}, \overline{X'_h}, \widetilde{X'_{h+1}}, \ldots, \widetilde{X'_K})$ may be discarded in Algorithm 3 in favor of another tuple $(\widetilde{X_1}, \widetilde{X_2}, \ldots, \widetilde{X_K}) \in \mathcal{H}_j^\epsilon$, where $\widetilde{X_k} = (\widetilde{A_k}, \widetilde{B_k}, \widetilde{C_k}, \widetilde{L_k})$ for $k = 1, 2, \ldots, K$. For this tuple, by (35)–(37) and the construction in Algorithm 3, we have

$$\begin{cases} \widetilde{A_k} \le \delta \widetilde{A'_k} \le \delta^j A'_k = \delta^j A_k \\ \widetilde{B_k} \le \delta \widetilde{B'_k} \le \delta^j B'_k = \delta^j B_k \\ \widetilde{L_k} = \overline{L'_k} = L'_k = L_k \end{cases} \quad \text{if} \quad k \in \{1, 2, \ldots, K\} \setminus \{h\}; \tag{38}$$

$$\begin{cases} \widetilde{A_h} \le \delta\overline{A'_h} = \delta(\widetilde{A'_h} + a_j) \le \delta(\delta^{j-1}A'_h + a_j) \le \delta^j(A'_h + a_j) = \delta^j A_h \\ \widetilde{B_h} \le \delta\overline{B'_h} = \delta(\widetilde{B'_h} + b_j) \le \delta(\delta^{j-1}B'_h + b_j) \le \delta^j(B'_h + b_j) = \delta^j B_h \\ \widetilde{L_h} = \overline{L'_h} = w_j = L_h \end{cases}; \tag{39}$$

$$\begin{cases} \widetilde{C_k} \le \delta\widetilde{C'_k} \le \delta^j C'_k = \delta^j C_k & \text{if } \ k \in \{1,2,\ldots,K-1\} \setminus \{h\} \\ \widetilde{C_K} \le \widetilde{C'_K} \le \delta^{j-1}C'_K \le \delta^j C_K & \text{if } \ K \ne h \end{cases}; \tag{40}$$

$$\begin{cases} \widetilde{C_h} \le \delta\overline{C'_h} = \delta(\max\{\widetilde{C'_h}, \widetilde{A'_h} + a_j\} + b_j) \\ \quad \le \delta(\max\{\delta^{j-1}C'_h, \delta^{j-1}A'_h + a_j\} + b_j) & \text{if } \ h \ne K \\ \quad \le \delta^j(\max\{C'_h, A'_h + a_j\} + b_j) = \delta^j C_h \\ \widetilde{C_K} \le \overline{C'_K} = \max\{\widetilde{C'_K}\widetilde{A'_K} + a_j\} + b_j \\ \quad \le \max\{\delta^{j-1}C'_K, \delta^{j-1}A'_K + a_j\} + b_j & \text{if } \ h = K \\ \quad \le \delta^{j-1}(\max\{C'_K, A'_K + a_j\} + b_j) \le \delta^j C_K \end{cases}. \tag{41}$$

By (38)-(41), (34) holds for j when J_j is assigned to the h-th schedule. Since h is arbitrary, this covers every possible assignment of J_j. Thus, the induction step is established for j. □

To analyze the approximation error and time complexity of Algorithm 3, we use the following two well-known inequalities. The first follows because $f(x) = x\ln x - (x-1)$ is increasing on $[1, +\infty)$ and $f(1) = 0$. The second follows because $g(x) = (1 + \frac{x}{n})^n - (1 + 2x)$ is convex on $[0, 1]$, with $g(0) = 0$ and $g(1) \le 0$.

Lemma 4.5. *For any real $x \ge 1$, $\ln x \ge (x-1)/x$.*

Lemma 4.6. *For any real $0 < x \le 1$, $(1 + x/n)^n \le 1 + 2x$.*

Theorem 4.2. *For fixed K, Algorithm 3 is an FPTAS for $F2 \ || \ C^w_{\max}$ with $O((n^{3K}/\epsilon^{3K-1})(\log P)^{3K-1})$ running time.*

Proof. We first bound the approximation error of the algorithm. Let π' be the approximate schedule returned by algorithm 3, and let π_{opt} be the optimal schedule returned by Algorithm 2. Denote by $(X_1^*, X_2^*, \ldots, X_K^*) \in \mathcal{H}_n$ the tuple corresponding to π_{opt}, where $X_k^* = (A_k^*, B_k^*, C_k^*, L_k^*)$ for $k = 1, 2, \ldots, K$. By (30) and (31), we have

$$C^w_{\max}(\pi_{\text{opt}}) = \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h} A_i^* + C_{h+1}^* + \sum_{i=h+2}^{k} B_i^*\Big)\Big\}. \tag{42}$$

By Lemma 4.4, there exists a tuple $(\widetilde{X_1^*}, \widetilde{X_2^*}, \ldots, \widetilde{X_K^*}) \in \mathcal{H}_n^\epsilon$ generated by Algorithm 3, with $\widetilde{X_k^*} = (\widetilde{A_k^*}, \widetilde{B_k^*}, \widetilde{C_k^*}, L_k^*)$, such that

$$\widetilde{A_k^*} \le \delta^n A_k^*, \quad \widetilde{B_k^*} \le \delta^n B_k^*, \quad \widetilde{C_k^*} \le \delta^n C_k^*, \quad k = 1, 2, \ldots, K. \tag{43}$$

Let $\widetilde{\pi_{\text{opt}}}$ be the schedule corresponding to $(\widetilde{X_1^*}, \widetilde{X_2^*}, \ldots, \widetilde{X_K^*})$. Again by (30) and (31), we have

$$C_{\max}^w(\widetilde{\pi_{\text{opt}}}) = \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h}\widetilde{A_i^*} + \widetilde{C_{h+1}^*} + \sum_{i=h+2}^{k}\widetilde{B_i^*}\Big)\Big\}. \tag{44}$$

Recall that $0 < \epsilon \le 1$ and $\delta = 1 + \epsilon/(2n)$. Then by Lemma 4.6 and (42)–(44), we have

$$\begin{aligned} C_{\max}^w(\widetilde{\pi_{\text{opt}}}) &\le \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h}\delta^n A_i^* + \delta^n C_{h+1}^* + \sum_{i=h+2}^{k}\delta^n B_i^*\Big)\Big\} \\ &= \delta^n \max_{1\le k\le K}\Big\{W_k \max_{0\le h\le k-1}\Big(\sum_{i=1}^{h} A_i^* + C_{h+1}^* + \sum_{i=h+2}^{k} B_i^*\Big)\Big\} \\ &\le (1+\epsilon) C_{\max}^w(\pi_{\text{opt}}). \end{aligned}$$

Thus, from the execution of Algorithm 3, it returns an approximate schedule π' with $C_{\max}^w(\pi') \le C_{\max}^w(\widetilde{\pi_{\text{opt}}}) \le (1+\epsilon) C_{\max}^w(\pi_{\text{opt}})$.

We next bound the time complexity of the algorithm. Step 1 sorts in $O(n \log n)$ time, and Step 2 initializes in $O(n)$ time. In Step 3, at most one tuple is retained per label after elimination. For each $(X_1, X_2, \ldots, X_K) \in \mathcal{H}_j^\epsilon$ $(j = 1, 2, \ldots, n)$, the corresponding label is $(Y_1, Y_2, \ldots, Y_K)$, where $X_k = (A_k, B_k, C_k, L_k)$ for $k = 1, 2, \ldots, K$, $Y_k = (\Delta(A_k), \Delta(B_k), \Delta(C_k), L_k)$ for $1 \le k \le K-1$, and $Y_K = (\Delta(A_K), \Delta(B_K), L_K)$. Note that $A_k \le P$, $B_k \le P$, $C_k \le 2P$, and $L_k \in \{0, W_k, W_{k+1}, \ldots, W_K\}$ for $k = 1, 2, \ldots, K$. By Lemma 4.5, the number of distinct values attained by $\Delta(A_k)$ is upper bounded by $1 + \lfloor \log_\delta P \rfloor = 1 + \lfloor \ln P / \ln \delta \rfloor \le 1 + \lfloor (1 + 2n/\epsilon) \ln P \rfloor = O((n/\epsilon) \log P)$. Analogously, the number of distinct values attained by $\Delta(B_k)$ and $\Delta(C_k)$ are also upper bounded by $O((n/\epsilon) \log P)$. Because each tuple yields at most K new tuples, after elimination, the number of retained tuples in $\mathcal{H}_j^\epsilon$ is upper bounded by $O((n^{3K-1}/\epsilon^{3K-1})(\log P)^{3K-1})$. Summing the cost of Step 3 over its n iterations gives $O((n^{3K}/\epsilon^{3K-1}) \log P)$ time. Step 4 computes the objective value of each tuple in $\mathcal{H}_n^\epsilon$ in $O(K)$ time, and thus requires $O((n^{3K-1}/\epsilon^{3K-1})(\log P)^{3K-1})$ time in total. Step 5 has the same time complexity. Consequently, the algorithm runs in $O((n^{3K}/\epsilon^{3K-1})(\log P)^{3K-1})$ time. □

4.4 The bounded max-to-min weight ratio

Consider the bounded max-to-min ratio weight case where $w_{\max}/w_{\min} \le C$ for some constant C. Here, $w_{\max} = \max_{1\le j\le n}\{w_j\}$ is the maximum weight of the n jobs, and $w_{\min} = \min_{1\le j\le n}\{w_j\}$ is the minimum weight. The proof of Theorem 3.1 demonstrates that the problem remains $\mathcal{NP}$-hard even when $C = 2$.

Let $0 < \epsilon \le 1$ be a real number. For $j = 1, 2, \ldots, n$, we round each weight w_j down to the nearest value of the form $w_{\min}(1+\epsilon/2)^k$, i.e., $\overline{w_j} = w_{\min}(1+\epsilon/2)^{\theta_j}$, where θ_j is the unique integer satisfying

$$w_{\min}(1+\epsilon/2)^{\theta_j} \le w_j < w_{\min}(1+\epsilon/2)^{\theta_j+1}. \tag{45}$$

By combining job-weight rounding with the FPTAS in Section 4.3, a PTAS for the bounded max-to-min ratio weight case is given as follows.

Algorithm 4: A PTAS for $F2 \,||\, C^w_{\max}$ with bounded max-to-min weight ratio

Input: Job set $\mathcal{J}$; error parameter ϵ; job parameters a_j, b_j, and w_j for $j = 1, 2, \ldots, n$.

1 **Step 1.** Construct the rounded instance $\mathcal{I}'$ from the original instance $\mathcal{I}$: replace each job J_j by J'_j with $w'_j = w_{\min}(1+\epsilon/2)^{\theta_j}$, where θ_j is the integer given by (45); keep all other job parameters unchanged.

2 **Step 2.** Let $W_1 > W_2 > \cdots > W_K$ be the ordered sequence of distinct rounded weights in $\mathcal{I}'$. Run Algorithm 3 on $\mathcal{I}'$ to obtain a $(1+\epsilon/3)$-approximation schedule π_ϵ.

Output: The approximate schedule π_ϵ.

For a schedule π, let $C^w_{\max}(\pi, \mathcal{I}) = \max_{1\le j\le n}\{w_{\pi(j)}C_{\pi(j)}(\pi, \mathcal{I})\}$ and $C^w_{\max}(\pi, \mathcal{I}') = \max_{1\le j\le n}\{w'_{\pi(j)}C_{\pi(j)}(\pi, \mathcal{I}')\}$ denote its weighted makespan on instances $\mathcal{I}$ and $\overline{\mathcal{I}}$, respectively. Here, $C_{\pi(j)}(\pi, \mathcal{I})$ and $C_{\pi(j)}(\pi, \mathcal{I}')$ are the completion times of $J_{\pi(j)}$ under π on $\mathcal{I}$ and $\mathcal{I}'$, respectively.

Theorem 4.3. *When $w_{\max}/w_{\min} \le C$, Algorithm 4 is a PTAS for $F2 \,||\, C^w_{\max}$ with $O((n^{O(1/\epsilon)}/\epsilon^{O(1/\epsilon)})(\log P)^{O(1/\epsilon)})$ running time.*

Proof. Let π_{opt} be an optimal schedule for $\mathcal{I}$, and let π'_{opt} be an optimal schedule for $\mathcal{I}'$. Because $\mathcal{I}$ and $\mathcal{I}'$ differ only in their job weights, a fixed schedule π yields the same job completion times on both instances, i.e., $C_{\pi(j)}(\pi, \mathcal{I}) = C_{\pi(j)}(\pi, \mathcal{I}')$. Thus, from the fact that $w'_j \le w_j < (1+\epsilon/2)w'_j$, $j = 1, 2, \ldots, n$, we have

$$C^w_{\max}(\pi_\epsilon, \mathcal{I}) < (1+\epsilon/2)C^w_{\max}(\pi_\epsilon, \mathcal{I}') \tag{46}$$

and

$$C^w_{\max}(\pi'_{\text{opt}}, \mathcal{I}') \le C^w_{\max}(\pi_{\text{opt}}, \mathcal{I}') \le C^w_{\max}(\pi_{\text{opt}}, \mathcal{I}). \tag{47}$$

Since π_ϵ is a $(1+\epsilon/3)$-approximation schedule for $\mathcal{I}'$, we have

$$C^w_{\max}(\pi_\epsilon, \mathcal{I}') \le (1+\epsilon/3)C^w_{\max}(\pi'_{\text{opt}}, \mathcal{I}'). \tag{48}$$

Then, (46)–(48) imply that

$$C^w_{\max}(\pi_\epsilon, \mathcal{I}) \le (1+\epsilon/2)(1+\epsilon/3)C^w_{\max}(\pi'_{\text{opt}}, \mathcal{I}') \le (1+\epsilon)C^w_{\max}(\pi_{\text{opt}}, \mathcal{I}).$$

Let $\mathcal{W} = \{\overline{w_j} : 1 \le j \le n\}$ be the set of distinct rounded job weights in $\mathcal{I}'$, and let $K = |\mathcal{W}|$. Since $w_{\max}/w_{\min} \le C$ and C is a constant, we have $K \le 1 + \lfloor \log_{1+\epsilon/2} C \rfloor = O(1/\epsilon)$. The running time is dominated by Step 2. By Theorem 4.2, it requires $O((n^{3K}/\epsilon^{3K-1})(\log P)^{3K-1}) = O((n^{O(1/\epsilon)}/\epsilon^{O(1/\epsilon)})(\log P)^{O(1/\epsilon)})$ time. Consequently, the overall running time is $O((n^{O(1/\epsilon)}/\epsilon^{O(1/\epsilon)})(\log P)^{O(1/\epsilon)})$. □

5 Heuristic and metaheuristic algorithms

In this section, we first present two constructive heuristics: the largest-weight-first (LW) greedy algorithm, whose worst-case approximation ratio is proved to be 2, and an adapted NEH heuristic that builds a schedule by inserting jobs sequentially. We then develop a genetic algorithm (GA), a metaheuristic that searches over job permutations.

5.1 The LW algorithm

Sun (2024) proved that the LW-based greedy algorithm is a $(2-1/m)$-approximation for $Pm \mid\mid C^w_{\max}$. For $F2 \mid\mid C^w_{\max}$, we describe the analogous LW-based algorithm and show that its worst-case approximation ratio is 2.

Algorithm 5: LW's rule for $F2 \mid\mid C^w_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j, b_j, and w_j for $j = 1, 2, \ldots, n$.

1 **Step 1.** Renumber the jobs in nonincreasing order of their weights (i.e., the LW order), so that $w_1 \geq w_2 \geq \cdots \geq w_n$.

2 **Step 2.** Process the jobs in this order on M_1 and then on M_2, yielding the permutation schedule π_{lw}.

Output: The resulting schedule π_{lw}.

Theorem 5.1. *For $F2 \mid\mid C^w_{\max}$, Algorithm 5 has a worst-case approximation ratio of 2, which is tight, and runs in $O(n \log n)$ time.*

Proof. Since $1 \mid\mid C^w_{\max}$ can be solved in $O(n \log n)$ time by the LW rule (Nong et al., 2011), we have

$$C^w_{\max}(\pi_{\mathrm{opt}}) \geq \mathrm{LB} = \max\left\{ \max_{1\leq j\leq n}\left\{w_j \sum_{i=1}^{j} a_i\right\}, \max_{1\leq j\leq n}\left\{w_j \sum_{i=1}^{j} b_i\right\}, \max_{1\leq j\leq n}\left\{w_j(a_j+b_j)\right\}\right\}, \quad (49)$$

where the first two terms are machine-based lower bounds and the last is a job-based lower bound.

By construction, Algorithm 5 yields $\pi_{\mathrm{lw}}(j) = j$ for $j = 1, 2, \ldots, n$. Combining (1) with (2), the weighted makespan of π_{lw} can be written as

$$C^w_{\max}(\pi_{\mathrm{lw}}) = \max_{1\leq j\leq n}\left\{w_{\pi_{\mathrm{lw}}(j)} C_{\pi_{\mathrm{lw}}(j)}\right\} = \max_{1\leq j\leq n}\left\{w_j \max_{1\leq i\leq j}\left(\sum_{h=1}^{i} a_h + \sum_{h=i}^{j} b_h\right)\right\}. \quad (50)$$

By (49) and (50), we have

$$\begin{aligned} C^w_{\max}(\pi_{\mathrm{lw}}) &\le \max_{1\le j\le n}\left\{w_j\sum_{h=1}^{j}a_h + w_j\sum_{h=1}^{j}b_h\right\} \\ &\le \max_{1\le j\le n}\left\{w_j\sum_{h=1}^{j}a_h\right\} + \max_{1\le j\le n}\left\{w_j\sum_{h=1}^{j}b_h\right\} \\ &\le 2C^w_{\max}(\pi_{\mathrm{opt}}). \end{aligned} \tag{51}$$

To show that the bound is tight, consider an instance with two jobs: J_1 has $a_1 = 1+\epsilon$, $b_1 = \epsilon$, and $w_1 = 1+\epsilon$, and J_2 has $a_2 = \epsilon$, $b_2 = 1+\epsilon$, and $w_2 = 1$, where $\epsilon > 0$ is arbitrarily small. Algorithm 5 yields the schedule $\pi_{\mathrm{lw}} = (1,2)$ with $C^w_{\max}(\pi_{\mathrm{lw}}) = 2+\epsilon$, while the optimal schedule is $\pi_{\mathrm{opt}} = (2,1)$ with $C^w_{\max}(\pi_{\mathrm{opt}}) = (1+\epsilon)(1+2\epsilon)$. Hence, $C^w_{\max}(\pi_{\mathrm{lw}})/C^w_{\max}(\pi_{\mathrm{opt}}) \to 2$ as $\epsilon \to 0$. □

5.2 The NEH algorithm

The NEH heuristic is a classical insertion-based constructive method for the permutation flow shop scheduling problem. Following Nawaz et al. (1983), jobs are inserted one at a time: at each iteration, the next job is tested in every feasible position of the current partial schedule, and the position yielding the minimum makespan is selected. NEH has proven competitive in flow-shop scheduling and remains widely used (Puka et al., 2024; Shao et al., 2023; Wang et al., 2024a,b). For the $C^w_{\max}$ objective, two observations motivate our adaptation. First, jobs with large $a_j + b_j$ tend to lengthen the schedule and should therefore be placed early to avoid delaying other jobs. Second, earlier positions reduce completion times, and this reduction has a greater impact on the objective for jobs with larger w_j. We therefore prioritize jobs by decreasing $a_j + b_j$, breaking ties by decreasing w_j, and insert each job into the position that minimizes the objective value. The procedure is detailed in Algorithm 6. At iteration k, evaluating all k insertion positions takes $O(k^2)$ time, so the overall running time is $O(n^3)$.

Algorithm 6: NEH heuristic for $F2 \,||\, C^w_{\max}$

Input: Job set $\mathcal{J}$; job parameters a_j, b_j, and w_j for $j = 1, 2, \ldots, n$.

Step 1. [Preprocessing] Renumber the jobs in $\mathcal{J}$ by nonincreasing $a_j + b_j$, and the ties are broken by nonincreasing w_j.

Step 2. [Initialization] $\pi \leftarrow (J_1)$.

Step 3. [Insertion] **for** $k = 2$ **to** n **do**

  $\pi \leftarrow$ insert J_k into π at a position that minimizes $C^w_{\max}$.

end

Output: The resulting schedule π.

5.3 The genetic algorithm

Genetic algorithms are widely used for scheduling problems, with operators designed around problem-specific structure. Recent flow-shop studies have tailored genetic operators to blocking constraints (de Athayde Prata and Fuchigami, 2024), sequence-independent setup times with non-interruption requirements (Qiao et al., 2022), and energy consumption under variable machine speeds (Wang et al., 2023). For the $C^w_{\max}$ objective, Lemma 4.1 implies that an optimal schedule is composed of weight-based blocks, with jobs inside each block ordered by Johnson's rule. Building on this structure, we develop a weight-block genetic algorithm (Algorithm 7) whose key component is a weight-class-based mutation operator.

Each chromosome encodes a job permutation. The initial population consists of the schedule generated by Johnson's rule and $N-1$ adjacent-swap variants of it, where N is the population size. Although Johnson's rule ignores job weights, it provides a structured, makespan-oriented schedule that serves as a starting point for the weight-aware search. Parents are chosen by fitness-based binary tournament. The offspring then undergo partially mapped crossover and weight-class-based mutation. Elitism retains the N best individuals from the combined parent–offspring pool.

The weight-class mutation operator is a key feature of Algorithm 7. Rather than moving individual jobs, it extracts an entire weight class, keeps its internal order, and reinserts the class as a block at the position minimizing $C^w_{\max}$. The class is chosen with equal probability either uniformly at random or from classes containing a job attaining the current $C^w_{\max}$. This block move exploits the problem structure. Since a forward move can delay the jobs it passes, it may not improve the schedule; selection then decides whether it is retained. A local search subsequently relocates the k jobs with the largest w_jC_j values, one at a time, to positions minimizing $C^w_{\max}$. It terminates after G rounds or when a round yields no improvement in $C^w_{\max}$. To balance solution quality against computation time, GA uses population size $N = \min\{20, \max\{6, \lceil n/2 \rceil\}\}$. For small instances, the generation count is $G = \max\{1, \lfloor (n-5)/5 \rfloor\}$ and the offspring count per generation is $\lambda = \max\{1, \lfloor \max\{0, n-20\}^2/20 \rfloor\}$; for large instances, $G = 1 + \lfloor (n-100)/50 \rfloor$ and $\lambda = \max\{3, \lfloor 3n^3/10^6 \rfloor\}$.

Algorithm 7: The weight-block genetic algorithm (WBGA) for $F2 \mid\mid C^w_{\max}$

Input: Job set $\mathcal{J}$; number of distinct weights K; population size N; generations G; offspring λ; probabilities p_c, p_m; reinsertion count k.

Step 1. [Initialization] $P_0 \leftarrow$ Johnson's schedule and $N-1$ adjacent-swap variants; evaluate.
Step 2. [Evolution] **for** $g = 1$ **to** G **do**
  Selection $M_g \leftarrow$ binary tournament on P_{g-1}.
  Crossover while $|O_g| < \lambda$ **do**
    Draw π_1, π_2 from M_g; with probability p_c obtain two children by block crossover, otherwise copy the parents; add them to O_g.
  end
  Mutation Apply block mutation to each child with probability p_m; evaluate.
  Survival $P_g \leftarrow$ the N best of $P_{g-1} \cup O_g$ by $C^w_{\max}$.
end
$\pi^* \leftarrow$ the best of P_G.
Step 3. [Local search] **repeat**
  $J_1, \ldots, J_k \leftarrow$ the k jobs of π^* with the largest $w_j C_j$.
  $\pi' \leftarrow$ best schedule from removing one J_i and reinserting it at one position.
  if $C^w_{\max}(\pi') < C^w_{\max}(\pi^*)$ **then**
    $\pi^* \leftarrow \pi'$
  end
until *no reinsertion lowers* $C^w_{\max}(\pi^*)$*, or* G *rounds have been reached*
Output: π^*.

6 Numerical experiments

In this section, we carry out a series of computational experiments to examine the performance of the proposed exact and heuristic methods. All algorithms were implemented in Python 3.13 and tested on a computer running Windows 11, equipped with an Intel Core Ultra 5 230F processor and 16 GB of RAM. The MILP model was solved using Gurobi 13.0 with a single thread.

6.1 Experimental setup

To enable a fair and systematic comparison, we divided the experiments into three groups and controlled the parameter settings within each group. Let $\mathrm{UD}[a, b]$ denote the uniform distribution over the consecutive integers $a, a+1, \ldots, b$. For each group, problem instances were generated randomly from the prescribed uniform distributions. Each configuration comprised 20 independently generated instances, and each instance had a time limit of 600 s.

Group 1 concerns only the exact DP algorithm in Section 4.2 and evaluates its running time in seconds (s), with $p_{\max} \in \{40, 80\}$ and $a_j, b_j \sim UD[1, p_{\max}]$. With two distinct weight values, $w_j \in \{1, 10\}$, the instance sizes are $n \in \{20, 40, 50, 60, 120, 140\}$; with three values, $w_j \in \{1, 10, 100\}$, they are $n \in \{10, 20, 30, 40\}$. These values only capture differences among jobs and do not affect the running time.

Groups 2 and 3 evaluate the three heuristic algorithms LW, NEH, and GA in Section 5 using running time in milliseconds (ms) and the relative percentage gap in (52). Group 2 uses small-scale instances ($n \in \{10, 15, \ldots, 55\}$), and Group 3 use large-scale instances ($n \in \{100, 120, \ldots, 400\}$), it is computed relative to a prescribed lower bound. In both groups, $a_j, b_j \sim UD[1, p_{\max}]$ and $w_j \sim UD[1, w_{\max}]$, where $p_{\max} \in \{40, 80\}$ and $w_{\max} \in \{30, 100\}$.

The relative percentage gap is

$$\text{gap}_A = \frac{z_A - z_{\text{base}}}{z_{\text{base}}} \times 100\%, \tag{52}$$

where z_A is the objective value returned by heuristic A, with $A \in \{LW, NEH, GA\}$, and z_{base} is the reference value. For small-scale instances, z_{base} is the optimal objective value of the MILP model; for large-scale instances, it is the lower bound LB given in (49).

6.2 Computational results

The DP is substantially faster when applicable. With 2 weight values, it certifies optimality for all 20 instances at each tested size up to 60 jobs, for 19 and 2 of the 20 instances at 120 jobs under $[1, 40]$ and $[1, 80]$, respectively, and for 16 of the 20 instances at 140 jobs (Table 2). With 3 weight values, however, its running time increases much more rapidly. All 20 instances are solved within the time limit at 10 and 20 jobs, with mean running times below 0.01 s at 10 jobs and 1.09 s and 1.59 s at 20 jobs under $[1, 40]$ and $[1, 80]$, respectively. At 30 jobs, the mean running time increases to 58.67 s and 91.51 s, approximately fifty times the corresponding 20-job values, and one instance under $[1, 80]$ reaches the 600 s time limit. At 40 jobs, all instances exceed the time limit, and none is solved to optimality; accordingly, the corresponding row is left blank in Table 3.

The $MILP$ model certifies optimality for almost all small instances, although its mean running time increases from 70 ms at 10 jobs to 110 000 ms at 55 jobs (Tables 4 to 7). On small instances, the three heuristics differ primarily in solution quality. With $w_{\max} = 30$, NEH achieves the smallest mean gap in most configurations, ranging from 0.8% to 5.0%, compared with 4.5% to 8.1% for LW. GA is the least accurate on small instances, although its mean gap decreases from 12.6% and 14.3% at 10 jobs to 4.2% and 3.5% at 55 jobs under the two processing-time ranges (Tables 4 to 5 and Figure 6). With $w_{\max} = 100$, NEH remains the most accurate, with mean gaps ranging from 1.1% to 5.2%, while LW ranges from 3.8% to 9.1%. In contrast, GA maintains mean gaps between 14.7% and 26.8%, with no substantial improvement as the number of jobs increases (Tables 6 to 7 and Figure 7).

In terms of computational cost, LW is the fastest, followed closely by NEH, whereas GA is considerably slower, with its mean running time increasing from 0.64 ms at 10 jobs to 1 000 ms at 55 jobs.

On large instances, the three heuristics differ primarily in how their solution quality is affected by the number of weight values. With $w_{\max} = 30$, LW achieves the smallest mean gap in most configurations, ranging from 1.4% to 4.4%, while the gap of NEH ranges from 3.9% to 6.0%. By contrast, GA performs worst at 100 jobs, with mean gaps of 18.3% and 16.0% under $[1, 40]$ and $[1, 80]$, respectively. Its mean gap nevertheless decreases to 0.6% and 0.8% at 400 jobs, allowing GA to outperform LW in solution quality from 220 jobs under $[1, 40]$ and from 240 jobs under $[1, 80]$ (Tables 8 to 9 and Figure 8). With $w_{\max} = 100$, LW remains the most accurate, with mean gaps ranging from 1.6% to 4.4%, while those of NEH range from 5.0% to 7.6%. In contrast, GA maintains mean gaps between 17.5% and 50.7%, and no longer achieves the accuracy advantage observed with 30 weight values (Tables 10 to 11 and Figure 9). This improvement in solution quality comes at a substantial computational cost: under $[1, 40]$, GA requires 128 000 ms at 400 jobs, compared with 0.146 ms for LW and 2 180 ms for NEH.

Overall, the DP certifies optimality for all instances with up to 60 jobs and two weight values, but fails to solve all instances at 40 jobs with three weight values. Among the heuristics, LW is the fastest and maintains a mean gap below 9.1% across all configurations, while NEH achieves the smallest mean gaps on most small instances. GA is most competitive on large instances with few weight values, where its mean gap decreases from 18.3% at 100 jobs to 0.6% at 400 jobs, at the cost of a running time approximately six orders of magnitude longer than that of LW.

Table 2: DP, weights $\in \{1, 10\}$

n	$a_j, b_j \in [1, 40]$			$a_j, b_j \in [1, 80]$		
	time (s)		opt.	time (s)		opt.
	avg	max		avg	max	
20	0.01	0.11	20/20	0.01	0.08	20/20
40	2.41	7.02	20/20	7.95	32.37	20/20
50	11.80	24.25	20/20	28.90	63.55	20/20
60	25.05	58.58	20/20	78.55	176.76	20/20
120	269.83	600.00	19/20	565.30	600.00	2/20
140	405.57	600.00	16/20	−−	−−	−−

Table 3: DP, weights $\in \{1, 10, 100\}$

n	$a_j, b_j \in [1, 40]$			$a_j, b_j \in [1, 80]$		
	time (s)		opt.	time (s)		opt.
	avg	max		avg	max	
10	0.00	0.01	20/20	0.00	0.00	20/20
20	1.09	4.21	20/20	1.59	6.56	20/20
30	58.67	417.68	20/20	91.51	600.00	19/20
40	−−	−−	−−	−−	−−	−−

Table 4: Small scale, $p_{\max} = 40$, weights $\in \{1, \ldots, 30\}$

n	MILP		LW				NEH				GA			
	time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	7.0e1	1.2e2	7.3	17.2	8.8e−3	2.5e−2	**3.4**	24.9	7.3e−2	7.9e−2	12.6	30.6	6.4e−1	7.2e−1
15	1.9e2	4.7e2	6.8	16.9	9.9e−3	1.9e−2	**2.1**	8.5	1.9e−1	2.0e−1	18.2	49.5	2.6e0	2.9e0
20	5.6e2	1.9e3	6.2	19.9	1.3e−2	2.3e−2	**2.7**	11.7	3.8e−1	4.1e−1	12.1	33.0	1.0e1	1.1e1
25	1.5e3	5.7e3	5.6	21.5	1.4e−2	2.5e−2	**3.7**	17.5	7.0e−1	7.4e−1	15.3	57.5	2.8e1	3.3e1
30	2.8e3	1.0e4	5.6	14.9	1.4e−2	2.6e−2	**3.9**	10.8	1.1e0	1.2e0	11.6	26.7	6.2e1	7.0e1
35	8.4e3	3.6e4	5.7	12.2	1.4e−2	2.4e−2	**3.2**	13.8	1.8e0	1.8e0	12.8	38.7	1.2e2	1.3e2
40	2.1e4	1.2e5	4.9	10.6	1.7e−2	3.0e−2	**2.4**	6.2	2.5e0	2.6e0	7.0	17.5	2.4e2	2.5e2
45	3.1e4	2.5e5	6.0	19.0	1.8e−2	3.8e−2	**3.7**	10.3	3.5e0	3.7e0	8.8	24.7	4.1e2	4.5e2
50	8.4e4	5.1e5	4.5	13.0	1.8e−2	3.1e−2	**3.6**	8.4	4.8e0	4.9e0	6.5	22.9	6.7e2	7.3e2
55	1.1e5	5.2e5	4.7	10.9	2.7e−2	4.7e−2	**3.1**	11.9	6.2e0	6.4e0	4.2	14.6	1.0e3	1.2e3

Table 5: Small scale, $p_{\max} = 80$, weights $\in \{1, \ldots, 30\}$

n	MILP		LW				NEH				GA			
	time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	5.1e1	1.0e2	8.1	17.5	8.5e−3	1.8e−2	**0.8**	5.7	7.2e−2	7.8e−2	14.3	47.5	6.1e−1	6.8e−1
15	2.9e2	1.8e3	7.7	19.7	9.5e−3	2.0e−2	**2.4**	14.5	1.9e−1	2.1e−1	15.9	37.5	2.6e0	2.9e0
20	4.5e2	1.8e3	4.5	11.8	1.2e−2	2.9e−2	**2.6**	9.6	3.8e−1	4.1e−1	16.3	39.1	1.0e1	1.2e1
25	1.6e3	4.6e3	4.7	16.2	1.2e−2	2.1e−2	**2.1**	7.9	7.0e−1	7.4e−1	15.7	37.6	2.8e1	3.0e1
30	3.2e3	9.5e3	5.6	11.7	1.5e−2	2.9e−2	**2.6**	13.6	1.2e0	1.2e0	14.4	34.2	6.1e1	6.6e1
35	8.7e3	4.1e4	7.0	18.1	1.8e−2	3.1e−2	**4.3**	7.9	1.8e0	1.8e0	11.5	27.5	1.2e2	1.3e2
40	2.4e4	1.5e5	6.3	20.8	1.8e−2	3.1e−2	**2.4**	7.6	2.5e0	2.6e0	5.5	15.9	2.5e2	2.7e2
45	6.8e4	3.8e5	4.8	12.3	1.6e−2	1.9e−2	**2.8**	11.9	3.5e0	3.7e0	5.3	13.1	4.2e2	4.5e2
50	7.7e4	4.8e5	6.2	12.4	1.9e−2	3.0e−2	**3.8**	11.1	4.7e0	4.8e0	4.2	12.0	6.6e2	7.4e2
55	1.4e5	4.7e5	5.0	11.4	2.2e−2	3.9e−2	5.0	16.2	6.2e0	6.5e0	**3.5**	14.7	1.1e3	1.2e3

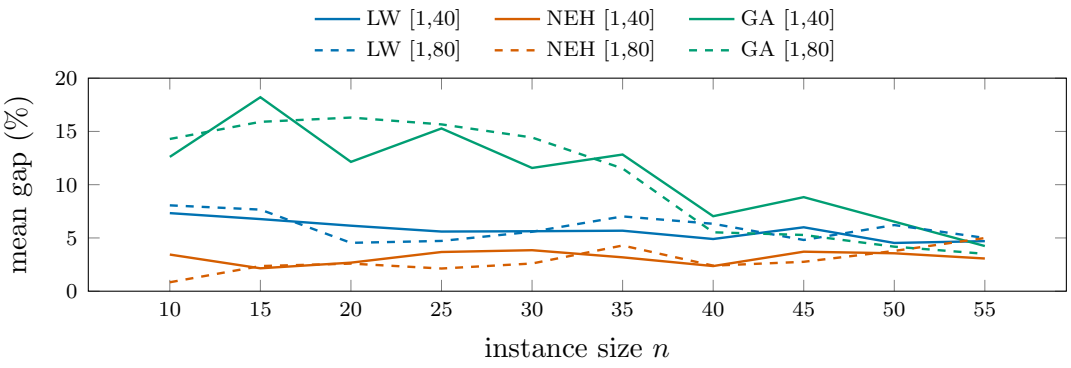


Figure 6: Small scale, weights $\in \{1, \ldots, 30\}$.

Table 6: Small scale, $p_{\max} = 40$, weights $\in \{1, \ldots, 100\}$

n	MILP		LW				NEH				GA			
	time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	7.0e1	1.4e2	6.5	23.9	8.2e−3	1.6e−2	**1.1**	9.0	7.4e−2	8.1e−2	20.6	60.4	6.4e−1	7.7e−1
15	2.0e2	4.4e2	6.8	29.3	1.0e−2	1.9e−2	**2.7**	22.9	1.9e−1	2.2e−1	21.2	51.3	2.6e0	3.0e0
20	3.8e2	6.2e2	4.7	19.8	1.1e−2	2.4e−2	**2.6**	11.0	3.9e−1	4.9e−1	20.1	42.0	1.1e1	1.2e1
25	1.3e3	2.9e3	6.6	16.7	1.3e−2	2.3e−2	**2.4**	9.2	7.1e−1	7.5e−1	24.8	52.9	2.9e1	3.6e1
30	4.6e3	1.9e4	6.7	13.0	1.4e−2	2.2e−2	**2.4**	7.5	1.1e0	1.2e0	23.5	58.9	6.3e1	6.8e1
35	1.1e4	5.4e4	5.4	14.5	1.6e−2	3.2e−2	**3.0**	10.9	1.7e0	1.8e0	26.8	59.2	1.3e2	1.4e2
40	2.2e4	1.1e5	5.4	15.8	1.7e−2	2.9e−2	**3.4**	10.4	2.6e0	2.7e0	21.3	51.6	2.6e2	3.1e2
45	3.5e4	2.1e5	4.9	12.9	1.9e−2	3.6e−2	**3.5**	11.9	3.5e0	3.8e0	20.3	36.9	4.2e2	4.6e2
50	8.2e4	3.9e5	4.8	14.9	2.0e−2	3.3e−2	**4.1**	10.0	4.8e0	4.9e0	21.0	44.3	6.9e2	7.3e2
55	1.1e5	4.1e5	4.5	14.9	2.1e−2	3.4e−2	**3.1**	6.9	6.3e0	6.4e0	17.7	31.9	1.1e3	1.1e3

Table 7: Small scale, $p_{\max} = 80$, weights $\in \{1, \ldots, 100\}$

n	MILP		LW				NEH				GA			
	time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	7.9e1	1.4e2	9.1	25.1	8.6e−3	1.6e−2	**2.2**	10.2	7.4e−2	8.8e−2	14.7	47.2	6.5e−1	7.6e−1
15	2.5e2	6.4e2	7.1	15.5	1.0e−2	1.7e−2	**1.8**	13.7	2.0e−1	2.5e−1	17.3	46.8	2.6e0	2.8e0
20	4.9e2	8.1e2	4.9	19.6	1.2e−2	2.1e−2	**1.6**	11.4	3.9e−1	4.3e−1	18.6	37.9	1.1e1	1.2e1
25	2.3e3	9.6e3	7.2	18.6	1.5e−2	3.0e−2	**3.1**	10.3	7.0e−1	7.5e−1	24.5	50.9	2.8e1	3.2e1
30	3.2e3	1.0e4	6.1	23.5	1.4e−2	2.7e−2	**2.6**	11.6	1.1e0	1.2e0	20.1	41.8	6.2e1	6.7e1
35	8.1e3	3.0e4	5.0	11.0	1.6e−2	2.7e−2	**2.7**	6.5	1.8e0	1.8e0	22.4	48.5	1.2e2	1.3e2
40	4.0e4	6.0e5	4.9	14.6	1.8e−2	3.0e−2	**4.3**	10.4	2.5e0	2.6e0	21.7	35.7	2.5e2	2.7e2
45	3.7e4	2.4e5	5.3	13.7	1.8e−2	4.2e−2	**4.2**	12.2	3.5e0	3.6e0	20.2	45.6	4.3e2	4.6e2
50	5.0e4	1.8e5	5.7	25.7	2.3e−2	3.6e−2	**5.2**	11.8	4.8e0	5.0e0	17.6	28.3	6.8e2	7.4e2
55	1.1e5	4.8e5	3.8	9.6	1.9e−2	3.4e−2	**3.4**	6.6	6.3e0	6.5e0	15.9	32.2	1.1e3	1.1e3

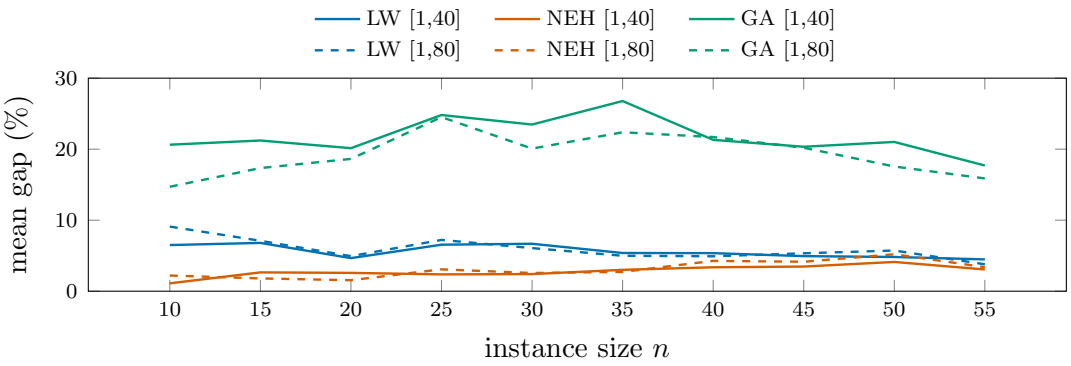


Figure 7: Small scale, weights $\in \{1, \ldots, 100\}$.

Table 8: Large scale, $p_{\max} = 40$, weights $\in \{1, \ldots, 30\}$

n	LW				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**4.4**	10.0	3.36e−2	6.16e−2	5.2	9.7	3.73e1	6.30e1	18.3	29.2	9.67e2	1.10e3
120	**3.5**	6.5	4.82e−2	1.05e−1	4.8	10.0	6.08e1	6.32e1	13.8	32.1	1.78e3	1.85e3
140	**3.0**	6.6	5.76e−2	1.19e−1	5.3	8.5	9.66e1	1.03e2	9.3	20.6	3.11e3	3.20e3
160	**3.4**	7.5	6.72e−2	1.28e−1	4.2	7.4	1.42e2	1.47e2	6.5	14.4	5.12e3	5.29e3
180	**2.6**	7.8	7.09e−2	9.77e−2	4.9	7.6	2.04e2	2.15e2	5.3	13.8	8.04e3	8.37e3
200	**2.6**	5.6	7.51e−2	1.17e−1	4.4	7.6	2.74e2	2.89e2	4.5	15.7	1.22e4	1.26e4
220	3.4	8.6	8.67e−2	1.05e−1	5.0	8.2	3.64e2	3.76e2	**3.0**	11.7	1.76e4	1.84e4
240	**2.8**	5.9	8.52e−2	1.11e−1	4.7	7.3	4.86e2	5.08e2	2.8	7.4	2.59e4	2.69e4
260	2.6	6.0	9.98e−2	1.48e−1	5.4	8.7	5.69e2	5.84e2	**1.6**	4.7	3.24e4	3.29e4
280	2.5	7.8	9.67e−2	1.20e−1	5.1	7.7	7.22e2	7.39e2	**0.8**	3.9	4.31e4	4.37e4
300	2.0	4.9	1.11e−1	1.79e−1	4.9	8.6	8.99e2	9.12e2	**0.9**	5.0	5.34e4	5.45e4
320	3.0	7.6	1.13e−1	1.23e−1	4.4	7.4	1.10e3	1.12e3	**1.2**	6.3	6.50e4	6.59e4
340	1.5	3.7	1.27e−1	2.41e−1	4.7	7.1	1.32e3	1.35e3	**1.2**	8.0	7.78e4	7.89e4
360	2.2	5.5	1.31e−1	1.58e−1	4.7	7.0	1.58e3	1.63e3	**0.9**	7.1	9.28e4	9.61e4
380	2.3	6.2	1.36e−1	1.50e−1	4.5	8.7	1.87e3	1.92e3	**0.3**	1.5	1.09e5	1.11e5
400	2.0	5.9	1.46e−1	1.69e−1	3.9	8.0	2.18e3	2.22e3	**0.6**	2.7	1.28e5	1.30e5

Table 9: Large scale, $p_{\max} = 80$, weights $\in \{1, \ldots, 30\}$

n	LW				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**3.6**	8.9	4.09e−2	1.25e−1	4.6	10.8	3.63e1	4.12e1	16.0	30.1	9.64e2	1.02e3
120	**4.4**	12.9	4.30e−2	7.13e−2	5.4	9.3	6.09e1	6.21e1	17.1	38.5	1.79e3	1.85e3
140	**3.5**	10.7	5.21e−2	9.38e−2	4.4	7.7	9.61e1	9.95e1	12.6	25.3	3.10e3	3.24e3
160	**3.2**	11.4	6.84e−2	1.12e−1	5.8	9.9	1.42e2	1.51e2	8.1	25.8	5.10e3	5.23e3
180	**3.0**	7.3	6.74e−2	9.56e−2	5.3	9.0	2.08e2	2.50e2	5.9	14.3	8.27e3	8.55e3
200	**2.7**	8.6	9.31e−2	2.55e−1	4.7	7.2	2.75e2	2.88e2	4.9	10.1	1.22e4	1.26e4
220	**2.9**	8.0	7.99e−2	1.12e−1	4.8	8.7	3.75e2	3.97e2	3.6	9.0	1.83e4	1.93e4
240	2.9	6.4	8.86e−2	1.17e−1	6.0	8.9	4.55e2	4.80e2	**1.9**	6.5	2.40e4	2.54e4
260	1.8	5.3	9.07e−2	1.07e−1	5.2	9.3	5.71e2	5.82e2	**1.6**	8.0	3.24e4	3.32e4
280	2.3	4.8	1.02e−1	1.27e−1	5.5	9.2	7.22e2	7.31e2	**0.8**	4.3	4.31e4	4.38e4
300	2.8	6.3	1.03e−1	1.23e−1	5.4	7.5	8.98e2	9.13e2	**2.0**	11.8	5.32e4	5.41e4
320	1.9	4.7	1.21e−1	2.31e−1	5.7	8.5	1.10e3	1.12e3	**1.4**	6.1	6.47e4	6.62e4
340	1.4	3.0	1.19e−1	1.33e−1	5.6	8.6	1.33e3	1.36e3	**0.8**	3.3	7.80e4	7.97e4
360	1.7	3.2	1.27e−1	1.51e−1	5.8	8.2	1.58e3	1.63e3	**1.1**	7.3	9.26e4	9.39e4
380	1.8	4.0	1.37e−1	1.56e−1	5.9	9.1	1.87e3	1.94e3	**0.8**	2.9	1.09e5	1.13e5
400	1.7	4.6	1.48e−1	2.07e−1	5.8	9.1	2.19e3	2.26e3	**0.8**	2.9	1.27e5	1.29e5

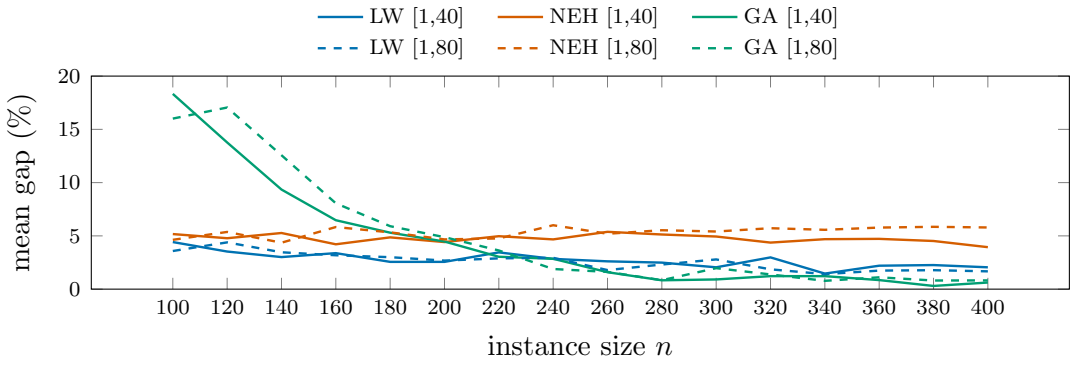


Figure 8: Large scale, weights $\in \{1, \ldots, 30\}$.

Table 10: Large scale, $p_{\max} = 40$, weights $\in \{1, \ldots, 100\}$

n	LW				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**4.0**	10.6	4.4e−2	9.6e−2	5.9	9.6	8.8e1	1.4e2	48.9	76.5	2.1e3	2.5e3
120	**3.3**	11.6	6.5e−2	1.8e−1	5.6	9.6	1.4e2	1.7e2	48.0	68.7	4.1e3	4.7e3
140	**2.9**	8.8	7.2e−2	3.0e−1	5.2	9.1	2.3e2	2.9e2	42.1	63.9	6.9e3	7.5e3
160	**3.1**	5.7	8.8e−2	2.3e−1	5.6	8.0	3.0e2	4.3e2	45.1	72.8	1.1e4	1.2e4
180	**3.5**	7.6	7.3e−2	1.9e−1	5.7	9.4	4.3e2	6.1e2	42.0	57.8	1.7e4	1.9e4
200	**3.2**	8.7	8.3e−2	1.4e−1	6.1	10.6	6.3e2	8.4e2	42.2	85.2	2.6e4	2.8e4
220	**2.5**	6.2	1.0e−1	1.5e−1	5.8	9.0	8.2e2	9.6e2	33.7	44.3	4.0e4	4.7e4
240	**2.6**	7.5	1.2e−1	2.0e−1	5.9	8.8	1.0e3	1.3e3	31.9	57.2	5.5e4	5.8e4
260	**2.8**	5.0	1.6e−1	3.1e−1	5.7	7.9	1.3e3	1.6e3	30.2	46.2	7.7e4	8.5e4
280	**2.2**	4.2	1.5e−1	3.5e−1	6.1	8.9	1.9e3	2.6e3	26.6	40.1	1.2e5	1.6e5
300	**2.2**	4.9	1.3e−1	2.6e−1	5.7	8.6	1.8e3	1.8e3	24.7	37.0	1.1e5	1.1e5
320	**1.9**	4.5	1.7e−1	4.2e−1	5.1	8.3	2.2e3	2.3e3	23.2	38.1	1.3e5	1.4e5
340	**1.6**	3.2	1.6e−1	2.8e−1	5.8	8.0	2.7e3	2.8e3	21.7	34.7	1.6e5	1.6e5
360	**1.9**	4.5	1.6e−1	2.6e−1	5.7	8.5	3.2e3	3.3e3	17.5	24.9	1.9e5	1.9e5
380	**1.7**	5.6	1.7e−1	2.6e−1	5.5	8.9	3.7e3	3.9e3	22.0	39.6	2.2e5	2.2e5
400	**1.6**	5.8	1.7e−1	3.2e−1	5.3	7.8	4.3e3	4.4e3	19.3	36.7	2.5e5	2.6e5

Table 11: Large scale, $p_{\max} = 80$, weights $\in \{1, \ldots, 100\}$

n	LW				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**4.3**	10.9	3.8e−2	6.9e−2	5.3	10.0	8.1e1	1.1e2	50.7	78.0	2.2e3	2.5e3
120	**4.4**	13.1	5.7e−2	9.8e−2	5.7	11.8	1.4e2	1.8e2	45.8	75.3	4.0e3	4.4e3
140	**3.1**	8.3	7.6e−2	2.4e−1	6.2	10.2	2.2e2	3.4e2	44.5	69.1	7.0e3	7.5e3
160	**3.0**	8.0	7.7e−2	1.4e−1	5.0	7.7	3.0e2	3.5e2	38.9	56.4	1.1e4	1.2e4
180	**3.2**	7.0	8.0e−2	2.3e−1	5.9	11.1	4.3e2	7.8e2	40.5	61.4	1.7e4	1.9e4
200	**3.4**	7.2	1.1e−1	3.3e−1	6.7	9.9	6.3e2	1.1e3	37.1	49.3	2.7e4	3.2e4
220	**3.0**	7.0	1.2e−1	2.4e−1	6.5	9.5	8.2e2	1.0e3	35.0	59.2	3.9e4	4.2e4
240	**2.4**	5.5	1.1e−1	2.0e−1	7.3	10.1	9.9e2	1.1e3	33.4	58.6	5.5e4	5.9e4
260	**2.3**	5.6	1.3e−1	3.0e−1	6.3	9.7	1.4e3	2.3e3	27.1	45.4	8.6e4	1.2e5
280	**1.7**	5.0	1.4e−1	2.2e−1	6.6	9.5	1.8e3	2.6e3	27.3	50.3	1.1e5	1.4e5
300	**2.7**	6.1	1.4e−1	2.1e−1	6.8	9.9	1.8e3	1.8e3	28.7	40.8	1.1e5	1.1e5
320	**2.4**	6.0	1.4e−1	2.5e−1	6.7	10.4	2.3e3	2.3e3	24.7	33.1	1.3e5	1.3e5
340	**2.1**	4.1	2.2e−1	1.5e0	6.5	9.3	2.7e3	2.8e3	22.9	38.5	1.6e5	1.6e5
360	**2.4**	5.6	1.9e−1	3.5e−1	7.6	10.0	3.2e3	3.4e3	21.3	40.3	1.9e5	1.9e5
380	**1.7**	5.3	1.8e−1	2.7e−1	5.5	8.5	3.7e3	3.8e3	18.5	37.0	2.2e5	2.2e5
400	**2.0**	4.5	2.0e−1	3.6e−1	6.6	9.0	4.2e3	4.4e3	18.6	30.6	2.6e5	2.7e5

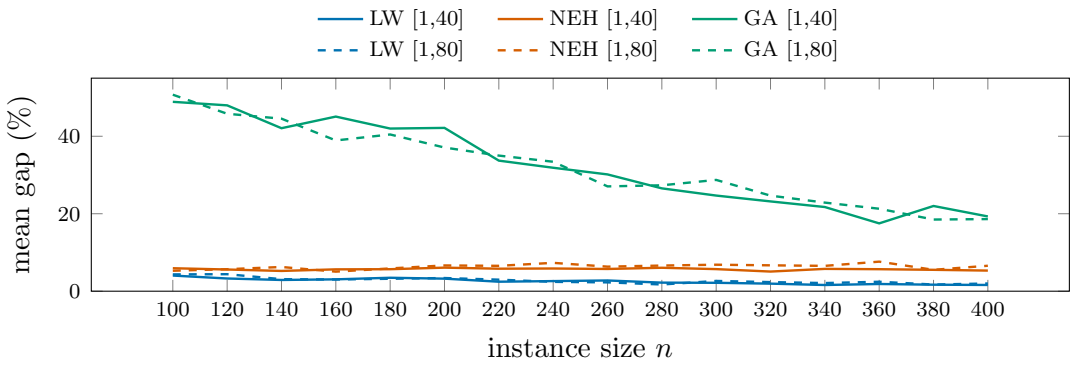


Figure 9: Large scale, weights $\in \{1, \ldots, 100\}$.

7 Conclusion

The maximum weighted completion time problem has a simple optimal rule on a single machine: jobs are scheduled in non-increasing order of weight. In a two-machine flow shop, however, a job's completion time depends on its position in the sequences processed on both machines, making the problem substantially more challenging. This paper studied the two-machine problem, $F2 \mid\mid WC_{\max}$, to identify the structural properties of optimal schedules and the algorithmic guarantees they enable.

We first established that the problem is binary $\mathcal{NP}$-hard even with only two distinct weights, and strongly $\mathcal{NP}$-hard when the number of distinct weights is unrestricted. We then showed that an optimal schedule consists of weight-based blocks ordered by non-increasing weight, with jobs within each block arranged according to Johnson's rule. This structure provides the basis for an exact DP that eliminates dominated states, an $FPTAS$ for a fixed number of distinct weights, and the LW rule. For larger instances, we adapted the NEH heuristic and developed a weight-block GA, and compared all these methods with the $MILP$ model.

The structural and computational limitations identified in this study motivate several directions for future research. First, both the block structure and the DP are developed for two machines. Extending the problem to an arbitrary number of machines requires investigating whether the block structure and ordering properties can be preserved, and how the role of Johnson's rule changes. Second, the running time of the DP grows rapidly with the number of distinct weight values, leaving some instances, such as those with 140 jobs under the wider processing-time range, unsolved. Whether a branch-and-bound algorithm can solve such instances more effectively remains an open question. Third, the heuristics considered here rely on manually designed rules and operators. Reinforcement learning, particularly Q-learning, could be used to learn dispatching rules for this objective or to guide the mutation and crossover operators of GA, potentially improving their adaptability to instance families not known in advance. More broadly, these results provide a structural

foundation for developing exact and heuristic methods for weighted bottleneck objectives in flow shops.

References

Agnetis, A., Billaut, J. C., Pinedo, M., and Shabtay, D. (2025). Fifty years of research in scheduling–Theory and applications. *European Journal of Operational Research*, 327(2):367–393.

Becker, T., Neufeld, J., and Buscher, U. (2025). The distributed flow shop scheduling problem with inter-factory transportation. *European Journal of Operational Research*, 322(1):39–55.

Chai, X., Lu, L., Li, W., and Zhang, L. (2018). Best-possible online algorithms for single machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 35(6):1850048.

de Athayde Prata, B. and Fuchigami, H. Y. (2024). A genetic iterated greedy algorithm for the blocking flowshop to minimize total earliness and tardiness. *Journal of Intelligent Manufacturing*, 35(5):2161–2174.

Della Croce, F. and Schau, Q. (2026). Revisiting Johnson's rule for minimizing makespan in the two-machine flow shop scheduling problem. *Operations Research Letters*, 69:107514.

Fan, B., Lu, L., and Shi, Z. (2026). On-line single-machine scheduling to minimize the weighted makespan under (dis)agreeability conditions. *Journal of Industrial and Management Optimization*, 22(8):4138–4150.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of $\mathcal{NP}$-Completeness*. W. H. Freeman and Company, San Francisco, CA.

Guo, S., Chen, Y., Mao, Y., and Zhang, X. (2024). Online single-processor scheduling with an unexpected breakdown. *Asia-Pacific Journal of Operational Research*, 41(1):2350005.

Hnaien, F. and Arbaoui, T. (2023). Minimizing the makespan for the two-machine flow shop scheduling problem with random breakdown. *Annals of Operations Research*, 328(2):1437–1460.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Li, W. (2015). A best possible online algorithm for the parallel-machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 32(04):1550030.

Li, W. and Liu, H. (2023). Online NDP-constraint scheduling of jobs with delivery times or weights. *Optimization Letters*, 17(3):591–612.

Liang, X., Lu, L., Sun, X., Yu, X., and Zuo, L. (2024). Online scheduling on a single machine with one restart for all jobs to minimize the weighted makespan. *AIMS Mathematics*, 9(1):2518–2529.

Liu, L. and Urgo, M. (2024a). A branch-and-bound approach to minimise the value-at-risk of the makespan in a stochastic two-machine flow shop. *International Journal of Production Research*, 62(6):2107–2123.

Liu, L. and Urgo, M. (2024b). Robust scheduling in a two-machine re-entrant flow shop to minimise the value-at-risk of the makespan: branch-and-bound and heuristic algorithms based on Markovian activity networks and phase-type distributions. *Annals of Operations Research*, 338(1):741–764.

Lu, L., Zuo, L., Zhang, L., and Ou, J. (2025). Order acceptance and scheduling with weighted makespan. *4OR*, 23(3):247–267.

Mosheiov, G. and Sarig, A. (2023). A note on lot scheduling on a single machine to minimize maximum weighted tardiness. *Journal of Combinatorial Optimization*, 45(5):128.

Nawaz, M., Enscore, E. E., and Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95.

Nong, Q. Q., Cheng, T. C. E., and Ng, C. T. (2011). Two-agent scheduling to minimize the total cost. *European Journal of Operational Research*, 215(1):39–44.

Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, Switzerland, 6 edition.

Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vN-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.

Qiao, Y., Wu, N., He, Y., Li, Z., and Chen, T. (2022). Adaptive genetic algorithm for two-stage hybrid flow-shop scheduling with sequence-independent setup time and no-interruption requirement. *Expert Systems with Applications*, 208:118068.

Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.

Shao, W., Shao, Z., and Pi, D. (2021). Effective constructive heuristics for distributed no-wait flexible flow shop scheduling problem. *Computers & Operations Research*, 136:105482.

Shao, W., Shao, Z., and Pi, D. (2023). Modelling and optimization of distributed heterogeneous hybrid flow shop lot-streaming scheduling problem. *Expert Systems with Applications*, 214:119151.

Smith, W. E. (1956). Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2):59–66.

Sun, R. (2024). Randomized approximation schemes for minimizing the weighted makespan on identical parallel machines. *Journal of Combinatorial Optimization*, 47:34.

Wang, B. T., Pan, Q. K., Gao, L., and Li, W. (2024a). The paradoxes, accelerations and heuristics for a constrained distributed flowshop group scheduling problem. *Computers & Industrial Engineering*, 196:110465.

Wang, C., Pan, Q. K., and Sang, H. Y. (2024b). The cascaded flowshop joint scheduling problem: a mathematical model and population-based iterated greedy algorithm to minimize total tardiness. *Robotics and Computer-Integrated Manufacturing*, 88:102747.

Wang, Y. J., Wang, G. G., Tian, F. M., Gong, D. W., and Pedrycz, W. (2023). Solving energy-efficient fuzzy hybrid flow-shop scheduling problem at a variable machine speed using an extended NSGA-II. *Engineering Applications of Artificial Intelligence*, 121:105977.

Yuan, J., Ng, C. T., and Cheng, T. C. E. (2020). Scheduling with release dates and preemption to minimize multiple max-form objective functions. *European Journal of Operational Research*, 280(3):860–875.

Zhang, H., Lu, L., and Yuan, J. (2025). Online scheduling on an unbounded parallel-batch machine to minimize the weighted makespan. *Journal of Combinatorial Optimization*, 49:6.

Zheng, F., Li, N., Liu, M., and Xu, Y. (2025). Single machine lot scheduling to minimize maximum weighted completion time. *Journal of Combinatorial Optimization*, 50:1.

Zuo, L., Lu, L., and Zhang, L. (2026a). Single machine scheduling with a restricted weight-modifying activity to minimize the weighted makespan. *International Journal of Foundations of Computer Science*, 37(5):693–707.

Zuo, L., Zhang, J., Shi, Z., and Fan, B. (2026b). Single machine scheduling with a restricted rate-modifying activity to minimize the weighted makespan. *Discrete Applied Mathematics*, 380:89–100.